

Rapport de Projet de Fin d'Études

AutoParts Pro

Réalisé par
Wiem Sayeh

Année Universitaire 2025-2026

Liste des acronymes

JWT	JSON Web Token
MSG	Management System Garage
JWT	JSON Web Token

Table des matières

Liste des acronymes	1
Liste des figures	6
Liste des tableaux	7
Introduction générale	8
1 Contexte général et spécification des exigences	10
1.1 Introduction	10
1.2 Présentation de l'organisme d'accueil	10
1.2.1 Quartis Technologies	10
1.2.2 Présentation du client MSG	11
1.3 Analyse des besoins	11
1.4 Étude de l'existant	11
1.4.1 Solutions e-commerce	12
1.4.2 Solutions ERP	12
1.4.3 Solutions spécialisées dans le secteur automobile	12
1.4.4 Comparaison des solutions existantes	12
1.5 Spécification des exigences	13
1.5.1 Identification des acteurs	13
1.5.2 Identification des exigences fonctionnelles	14
1.5.3 Identification des exigences non fonctionnelles	16
1.6 Méthodologie de développement	17
1.6.1 Méthodologie Scrum	18
1.6.2 Les rôles au sein de l'équipe Scrum	18
1.6.3 Les éléments de gestion de Scrum	19
1.6.4 Déroulement d'un Sprint	19
1.6.5 Langage de modélisation UML	20
1.7 Conclusion	21

2	Pilotage du projet et conception générale	22
2.1	Introduction	22
2.2	Pilotage du projet avec Scrum	22
2.2.1	Product Backlog	23
2.2.2	Planification des Sprints	24
2.3	Diagramme de cas d'utilisation global	25
2.4	Choix techniques	26
2.4.1	Environnement logiciel	26
2.5	Outils de développement	27
2.5.1	Environnement de développement	27
2.5.2	Outil de construction	28
2.5.3	Outils de gestion de versions	28
2.5.4	Outil de gestion du projet	28
2.6	Outils de conception et de modélisation	28
2.7	Architecture de l'application	28
2.7.1	Couche présentation	29
2.7.2	Couche métier	29
2.7.3	Couche données	29
2.8	Architecture générale des échanges	30
2.9	Conclusion	30
3	Réalisation du Sprint 1 : Gestion des utilisateurs et Authentification	32
3.1	Introduction	32
3.2	Analyse du Sprint	32
3.2.1	Sprint Backlog	32
3.3	Conception	34
3.3.1	Diagramme de Cas d'Utilisation	34
3.4	Description des Cas d'Utilisation	35
3.4.1	Diagrammes de Séquence	42
3.4.2	Diagramme de Classes	44
3.5	Architecture de Sécurité	46
3.5.1	Architecture Spring Security	48
3.5.2	Authentification basée sur JWT	49
3.6	Réalisation	50
3.6.1	Inscription d'un Customer et vérification du compte par e-mail . . .	50
3.6.2	Authentification des utilisateurs	51
3.6.3	Réinitialisation du mot de passe	51
3.6.4	Création et initialisation d'un Employee	53
3.7	Bilan du Sprint	53

4	Réalisation du Sprint 2 : Gestion des produits et du stock	55
4.1	Introduction	55
4.2	Analyse du Sprint	55
4.2.1	Sprint Backlog	55
4.3	Conception	57
4.3.1	Diagramme de Cas d'Utilisation	57
4.3.2	Diagramme de Séquence	61
4.3.3	Diagramme de classes	64
4.4	Réalisation	68
4.4.1	Gestion des produits	68
4.4.2	Gestion du stock	68
4.4.3	Gestion des fournisseurs	68
4.4.4	Gestion du panier et des commandes	68
4.4.5	Gestion des factures et des paiements	68
4.5	Bilan du Sprint	68
5	Réalisation du Sprint 3 : Gestion des services de maintenance et des interventions	69
5.1	Introduction	69
5.2	Analyse du Sprint	70
5.2.1	Sprint Backlog	70
5.3	Conception	72
5.3.1	Diagramme de Cas d'Utilisation	72
5.3.2	Diagramme de Séquence	76
5.3.3	Diagramme de Classes	81
5.4	Réalisation	82
5.4.1	Gestion des services de maintenance	82
5.4.2	Gestion des véhicules clients	83
5.4.3	Gestion des demandes de maintenance	84
5.4.4	Planification et affectation des interventions	85
5.4.5	Gestion du cycle de vie des demandes	86
5.4.6	Intégration avec l'e-commerce et l'ERP	86
5.5	Bilan du Sprint	87
6	Réalisation du Sprint 4 : Gestion des Ressources Humaines	89
6.1	Introduction	89
6.2	Analyse du Sprint	89
6.2.1	Sprint Backlog	89
6.3	Conception	92
6.3.1	Diagramme de Cas d'Utilisation	92

6.3.2	Diagramme de Séquence : Affectation automatique d'un employé . .	96
6.3.3	Diagramme de Classes	96
6.4	Architecture et intégration des services	97
6.4.1	Gestion des horaires et des shifts	97
6.4.2	Gestion des congés	97
6.4.3	Gestion des avances sur salaire	98
6.5	Réalisation	98
6.5.1	Vérification d'un Employee par CIN	98
6.5.2	Gestion des horaires de travail	98
6.5.3	Gestion des shifts	99
6.5.4	Gestion des demandes de congé	99
6.5.5	Gestion des avances sur salaire	99
6.6	Bilan du Sprint	99
7	Intégration et automatisation avec n8n	101
7.1	Introduction	101
7.2	Présentation et analyse du Sprint	101
7.2.1	Présentation de n8n	101
7.2.2	Objectifs du Sprint	102
7.2.3	Sprint Backlog	102
7.3	Intégration de n8n	103
7.3.1	Architecture de l'intégration	103
7.3.2	Communication entre Spring Boot et n8n	103
7.3.3	Gestion des Webhooks	103
7.4	Réalisation	104
7.4.1	Automatisation de la confirmation des commandes	104
7.4.2	Automatisation des paiements de maintenance	104
7.4.3	Gestion des alertes de stock faible	104
7.4.4	Automatisation de la réception des livraisons	105
7.4.5	Notification des demandes de congé	105
7.5	Bilan du Sprint	106
	Conclusion Générale	107
	Bibliographie	108

Table des figures

3.1	Diagramme de cas d'utilisation global du Sprint 1	35
3.2	Diagramme de séquence - Création d'un employé	43
3.3	Diagramme de séquence - Première authentification de l'employé	44
3.4	Diagramme de classes - Gestion des utilisateurs	46
3.5	Architecture de sécurité basée sur Spring Security et JWT	48
3.6	Interface d'inscription d'un Customer	50
3.7	Interface d'authentification Client	51
3.8	Interface d'authentification Admin et employee	51
3.9	Interface « Forgot Password «client» »	52
3.10	Interface « Forgot Password «admin et employee» »	52
3.11	Interface de réinitialisation du mot de passe client	52
3.12	Interface de réinitialisation du mot de passe admin et employee	53
3.13	Interface de création d'un Employee	53
4.1	Diagramme de cas d'utilisation du Sprint 2	59
4.2	Diagramme de séquence - Réception d'une livraison et mise à jour du stock	62
4.3	Diagramme de séquence - Ajustement manuel du stock	63
4.4	Diagramme de séquence - Ajout au panier et confirmation d'une commande	64
4.5	Diagramme de classes du sprint 2 - Gestion des produits	66
5.1	Diagramme de cas d'utilisation du Sprint 3	74
5.2	Diagramme de séquence - Création d'une demande de maintenance	77
5.3	Diagramme de séquence - Affectation automatique et planification d'une intervention	79
5.4	Diagramme de séquence - Traitement et suivi d'une intervention	80
5.5	Diagramme de classes du Sprint 3 - Gestion des services de maintenance	82

Liste des tableaux

1.1	Comparaison des solutions existantes avec la solution proposée	13
2.1	Product Backlog du projet AutoParts Pro	24
2.2	Planification des Sprints	25
3.1	Sprint Backlog - Gestion des utilisateurs et sécurité	34
3.2	Description du cas d'utilisation « Se connecter »	36
3.3	Description du cas d'utilisation « Oublier le mot de passe »	36
3.4	Description du cas d'utilisation « Réinitialiser le mot de passe »	37
3.5	Description du cas d'utilisation « Envoyer un e-mail de réinitialisation »	37
3.6	Description du cas d'utilisation « Créer un employé »	38
3.7	Description du cas d'utilisation « Envoyer les identifiants de connexion »	38
3.8	Description du cas d'utilisation « Attribuer un département »	39
3.9	Description du cas d'utilisation « Gestion des rôles et des permissions »	39
3.10	Description du cas d'utilisation « Changer le mot de passe »	40
3.11	Description du cas d'utilisation « S'inscrire »	40
3.12	Description du cas d'utilisation « Envoyer un e-mail de vérification »	41
3.13	Description du cas d'utilisation « Vérifier l'adresse e-mail »	42
4.1	Sprint Backlog du Sprint 2	57
5.1	Sprint Backlog du Sprint 3	72
6.1	Sprint Backlog - Gestion des Ressources Humaines	92
7.1	Sprint Backlog du Sprint 5	102

Introduction générale

Contexte

La transformation numérique constitue aujourd’hui un levier important pour améliorer la gestion et l’efficacité des entreprises. Dans le secteur automobile, la diversité des activités, allant de la commercialisation des pièces à la réalisation des services de maintenance, nécessite des outils permettant de centraliser les informations et de faciliter le suivi des opérations.

Dans ce contexte, ce projet a été réalisé pour **MSG**, un garage spécialisé dans la vente de pièces automobiles et les services de maintenance. L’objectif est de proposer une solution digitale adaptée à ses besoins et destinée à être intégrée à son activité.

Problématique

La gestion simultanée des produits, des stocks, des fournisseurs, des commandes, des clients et des interventions de maintenance peut devenir complexe lorsque les informations sont réparties entre plusieurs outils ou traitées manuellement.

Cette situation peut entraîner des difficultés dans le suivi des opérations, la circulation des informations et la coordination entre les différents intervenants. Il devient donc nécessaire de disposer d’une plateforme centralisée permettant d’unifier les principaux processus de l’entreprise.

Objectifs

L’objectif principal de ce projet est de concevoir et développer une plateforme permettant à MSG de centraliser ses activités commerciales et internes.

La solution doit notamment permettre de :

- gérer les utilisateurs, les rôles et les autorisations ;
- gérer les produits, les stocks et les fournisseurs ;
- gérer les commandes, les factures et les paiements ;
- gérer les services de maintenance et les interventions ;

- gérer les employés et certaines opérations liées aux ressources humaines ;
- automatiser certaines tâches et notifications ;
- proposer une interface e-commerce destinée aux clients.

Solution proposée

Afin de répondre aux besoins identifiés, nous avons développé **AutoParts Pro**, une plateforme combinant les fonctionnalités d'un **site e-commerce** et d'un **ERP** destiné à la gestion interne de MSG.

La solution permet de centraliser les processus liés à la vente de pièces automobiles, à la gestion des stocks et des fournisseurs, aux commandes et aux paiements, ainsi qu'aux services de maintenance. Elle intègre également un mécanisme de gestion des utilisateurs et des autorisations afin de sécuriser l'accès aux différentes fonctionnalités.

L'objectif est ainsi de fournir à MSG une solution évolutive permettant d'améliorer l'organisation de ses activités tout en offrant aux clients une expérience numérique adaptée.

Organisation du rapport

Ce rapport présente les différentes étapes suivies pour l'analyse, la conception et la réalisation de la solution.

Le premier chapitre présente le contexte général du projet, l'organisme d'accueil, le client MSG, l'analyse de l'existant ainsi que les exigences fonctionnelles et non fonctionnelles. Il présente également les acteurs du système et les choix méthodologiques adoptés.

Le deuxième chapitre est consacré au pilotage du projet et à la conception générale de la solution. Il présente notamment l'organisation des Sprints, le Product Backlog ainsi que l'architecture globale du système.

Les chapitres suivants présentent la réalisation des différents Sprints du projet. Ils couvrent progressivement l'authentification et la gestion des utilisateurs, la gestion des produits et des stocks, les services de maintenance, les ressources humaines ainsi que les mécanismes d'automatisation.

Enfin, une conclusion générale présente les principaux résultats obtenus, les limites éventuelles de la solution et les perspectives d'évolution.

Chapitre 1

Contexte général et spécification des exigences

1.1 Introduction

Ce chapitre présente le contexte général du projet ainsi que les éléments nécessaires à la compréhension et à la définition de la solution proposée. Il commence par la présentation de l'organisme d'accueil et du client MSG, puis décrit les besoins identifiés à travers une étude de l'existant.

Les exigences fonctionnelles et non fonctionnelles sont ensuite présentées afin de définir les principales fonctionnalités et contraintes auxquelles la solution doit répondre. Enfin, les acteurs du système, la méthodologie de développement ainsi que le langage de modélisation adoptés sont introduits.

1.2 Présentation de l'organisme d'accueil

1.2.1 Qartis Technologies

Qartis Technologies est une startup tunisienne créée en 2024, spécialisée dans le développement de solutions digitales modernes et dans l'accompagnement des entreprises dans leur transformation numérique. L'entreprise intervient notamment dans la conception d'applications web et mobiles adaptées aux besoins spécifiques de ses clients.

Elle s'appuie sur une équipe composée notamment de développeurs, de designers UI/UX et de chefs de projet, ainsi que sur une stack technologique diversifiée comprenant Angular, Flutter, NestJS, Python, Laravel et Firebase.

Qartis Technologies adopte une organisation agile favorisant le développement itératif, la collaboration entre les différents intervenants et l'adaptation continue aux besoins des clients. Elle a notamment participé à la réalisation de plateformes digitales telles qu'une

solution de réservation intelligente pour le padel et une marketplace permettant la mise en relation entre prestataires et clients.

Le projet présenté dans ce rapport a été réalisé dans cet environnement orienté vers l'innovation et le développement de solutions digitales personnalisées.

1.2.2 Présentation du client MSG

MSG est un garage automobile spécialisé dans la vente de pièces automobiles et dans la réalisation de services de maintenance. L'entreprise souhaite moderniser ses processus de gestion et améliorer l'expérience proposée à ses clients à travers une solution digitale centralisée.

Le projet réalisé vise ainsi à répondre aux besoins spécifiques de MSG en regroupant les fonctionnalités liées au commerce de pièces automobiles, à la gestion interne de l'entreprise et aux services de maintenance au sein d'une même plateforme.

1.3 Analyse des besoins

L'analyse des besoins a été réalisée à partir des activités de MSG et des différentes catégories d'utilisateurs amenées à utiliser la solution.

Plusieurs domaines fonctionnels ont ainsi été identifiés :

- gestion des utilisateurs et des autorisations ;
- gestion des produits, des catégories, des marques et des fournisseurs ;
- gestion des stocks et des approvisionnements ;
- gestion des ventes, des commandes et des paniers ;
- gestion des factures et des paiements ;
- gestion des services de maintenance et des interventions ;
- gestion des employés, des horaires, des shifts et des congés ;
- automatisation de certaines opérations métier.

Cette analyse constitue la base de la définition des fonctionnalités du système et permet d'organiser leur développement progressivement à travers les différents Sprints.

1.4 Étude de l'existant

Avant la conception de la solution, une étude de l'existant a été menée afin d'analyser les solutions actuellement disponibles sur le marché et d'identifier leurs avantages ainsi que leurs limites par rapport aux besoins spécifiques de MSG.

Trois catégories principales de solutions ont été étudiées : les plateformes e-commerce généralistes, les ERP traditionnels et les solutions spécialisées dans le secteur automobile.

1.4.1 Solutions e-commerce

Les plateformes e-commerce telles que WooCommerce permettent de créer rapidement des boutiques en ligne et proposent un grand nombre d’extensions pour ajouter différentes fonctionnalités.

Cependant, leur utilisation dans un contexte professionnel complexe peut nécessiter l’installation de nombreux plugins, ce qui peut entraîner des difficultés de maintenance, de personnalisation et de performance.

1.4.2 Solutions ERP

Les ERP tels qu’Odoo ou ERPNext permettent de centraliser plusieurs processus métiers, notamment la gestion des stocks, des fournisseurs, des ventes et des ressources humaines.

Cependant, leur configuration peut être relativement complexe et leur personnalisation nécessite parfois des connaissances spécifiques. Leur interface peut également être moins adaptée aux besoins d’une plateforme e-commerce destinée aux clients externes.

1.4.3 Solutions spécialisées dans le secteur automobile

Des solutions telles que TecDoc, AutoZone ou PartSouq proposent des fonctionnalités adaptées au domaine automobile, notamment la recherche et la compatibilité entre les pièces et les véhicules.

Toutefois, ces solutions sont généralement propriétaires et peuvent présenter des limitations en matière de personnalisation et d’intégration avec les processus internes spécifiques d’un garage.

1.4.4 Comparaison des solutions existantes

Afin de mieux mettre en évidence les limites des solutions étudiées, une comparaison a été réalisée selon plusieurs critères fonctionnels et techniques.

Critère	E-commerce	ERP	Solutions auto	Solution proposée
Gestion e-commerce	✓	Limitée	✗	✓
Gestion ERP	✗	✓	✗	✓
Gestion des stocks et fournisseurs	Limitée	✓	✗	✓
Compatibilité véhicule/pièce	✗	✗	✓	✓
Services de maintenance	✗	Limitée	Limitée	✓
Fonctionnalités intelligentes	✗	✗	✗	✓
Facilité d'intégration	Moyenne	Faible	Faible	Élevée

TABLE 1.1 – Comparaison des solutions existantes avec la solution proposée

Cette comparaison montre qu'aucune des solutions étudiées ne répond simultanément à l'ensemble des besoins identifiés pour MSG. Les plateformes e-commerce sont principalement orientées vers la vente en ligne, tandis que les ERP privilégient la gestion interne de l'entreprise. Les solutions automobiles spécialisées proposent quant à elles des fonctionnalités spécifiques au domaine, mais restent généralement limitées en matière de personnalisation et d'intégration.

La solution proposée vise donc à regrouper ces différents besoins au sein d'une plateforme intégrée combinant **e-commerce**, **ERP**, **gestion des services de maintenance** et **fonctionnalités intelligentes**.

1.5 Spécification des exigences

Dans cette section, nous procédons à l'identification des acteurs de notre système ainsi qu'à la spécification des exigences fonctionnelles et non fonctionnelles. Cette étape permet de déterminer les différents utilisateurs susceptibles d'interagir avec la plateforme ainsi que les fonctionnalités et contraintes auxquelles la solution doit répondre.

1.5.1 Identification des acteurs

L'identification des acteurs constitue une étape essentielle dans la délimitation du système. Un acteur représente un rôle joué par une entité externe qui interagit directement avec le système en échangeant des informations et en utilisant les fonctionnalités qui lui sont autorisées.

Notre solution comporte principalement quatre acteurs susceptibles d'interagir avec le système :

- **Customer** : représente le client du garage. Il peut créer un compte, consulter les produits disponibles, rechercher des pièces adaptées à son véhicule, gérer son panier,

passer des commandes et effectuer des paiements. Il peut également consulter les services de maintenance proposés par MSG et effectuer des réservations pour une intervention.

- **Employee** : représente un employé du garage. Son accès au système dépend de son rôle et de son département. Il peut effectuer les opérations métier correspondant à ses responsabilités, notamment la gestion des produits, des stocks, des ventes, des demandes de maintenance ou encore des ressources humaines.
- **Admin** : représente l'administrateur de la plateforme. Il possède les privilèges nécessaires pour administrer le système, gérer les comptes des employés, attribuer les rôles et les départements, gérer les permissions et superviser les différentes ressources de l'application.
- **Système** : représente les mécanismes automatiques de la plateforme. Il intervient notamment dans l'envoi des e-mails de vérification et de réinitialisation du mot de passe, la génération des tokens d'authentification, certaines notifications ainsi que l'exécution de traitements automatisés.

1.5.2 Identification des exigences fonctionnelles

Les exigences fonctionnelles décrivent les services et les fonctionnalités que le système doit fournir à ses différents utilisateurs. Elles sont définies à partir des besoins exprimés par MSG et des différents processus métiers identifiés.

Les principales exigences fonctionnelles de notre système sont présentées ci-dessous.

Fonctionnalités destinées au Customer

Après une authentification valide, le **Customer** doit pouvoir :

- créer un compte utilisateur en renseignant ses informations personnelles ;
- vérifier son adresse e-mail à partir du lien de confirmation reçu après son inscription ;
- se connecter à son compte à l'aide de ses identifiants ;
- demander la réinitialisation de son mot de passe en cas d'oubli ;
- définir un nouveau mot de passe à partir du lien sécurisé reçu par e-mail ;
- consulter les produits disponibles sur la plateforme ;
- rechercher et filtrer les produits selon différents critères tels que la catégorie, la marque ou les caractéristiques du produit ;
- rechercher des pièces compatibles avec son véhicule ;
- consulter les informations détaillées d'un produit ;
- ajouter des produits à son panier ;
- modifier les quantités des produits présents dans son panier ;
- passer une commande à partir des produits sélectionnés ;

- consulter l'historique et l'état de ses commandes ;
- consulter les informations relatives aux factures et aux paiements associés à ses commandes ;
- consulter les services de maintenance proposés par MSG ;
- réserver un service de maintenance en renseignant les informations nécessaires concernant son véhicule et l'intervention souhaitée ;
- consulter l'état de ses demandes et réservations de maintenance.

Fonctionnalités destinées à l'Employee

Après une authentification valide, l'**Employee** doit pouvoir accéder aux fonctionnalités correspondant à son rôle et à son département.

Selon les autorisations qui lui sont attribuées, il doit pouvoir :

- consulter les informations nécessaires à l'exécution de ses tâches ;
- gérer les produits et leurs informations ;
- consulter et gérer les niveaux de stock ;
- gérer les fournisseurs et les opérations d'approvisionnement ;
- gérer les ventes et les commandes selon ses permissions ;
- gérer les demandes de maintenance et les interventions techniques ;
- consulter les informations relatives aux clients nécessaires à son activité ;
- consulter et gérer les informations des employés selon les droits accordés ;
- consulter et gérer les congés, horaires et shifts selon son département ;
- accéder uniquement aux fonctionnalités autorisées par son rôle et son département.

Les fonctionnalités accessibles à l'employé sont ainsi contrôlées par le mécanisme d'autorisation mis en place dans l'application.

Fonctionnalités destinées à l'Admin

Après une authentification valide, l'**Admin** doit pouvoir administrer l'ensemble de la plateforme.

Il doit notamment pouvoir :

- gérer les comptes des employés ;
- créer un compte Employee ;
- modifier les informations d'un employé ;
- attribuer un rôle à un employé ;
- affecter un département à un employé ;
- gérer les permissions associées aux différents rôles ;
- consulter la liste des utilisateurs et leurs informations ;
- activer ou désactiver les comptes selon les besoins ;
- gérer les départements de l'entreprise ;

- consulter les informations nécessaires à la supervision du système ;
- accéder aux différentes fonctionnalités administratives de la plateforme.

Fonctionnalités automatiques du système

Le **système** doit également assurer plusieurs traitements automatiques nécessaires au bon fonctionnement de la plateforme.

Il doit notamment pouvoir :

- créer automatiquement le compte administrateur principal lors du premier démarrage lorsque aucun administrateur n'existe ;
- générer et envoyer un e-mail de vérification lors de l'inscription d'un Customer ;
- générer un token de vérification permettant d'activer un compte ;
- envoyer un e-mail contenant un lien sécurisé lors d'une demande de réinitialisation du mot de passe ;
- générer les identifiants de connexion lors de la création d'un Employee ;
- envoyer les informations de connexion au nouvel Employee par e-mail ;
- générer un token JWT après une authentification réussie ;
- vérifier la validité du token JWT lors de l'accès aux ressources protégées ;
- contrôler les autorisations de l'utilisateur en fonction de son rôle et, pour les employés, de son département ;
- exécuter les traitements automatiques nécessaires au fonctionnement des différents modules de la plateforme.

1.5.3 Identification des exigences non fonctionnelles

Les exigences non fonctionnelles définissent les contraintes de qualité et les propriétés que la solution doit respecter afin d'assurer son bon fonctionnement, sa sécurité et sa facilité d'utilisation.

Pour garantir la qualité de la plateforme MSG, les contraintes suivantes doivent être respectées :

- **Ergonomie** : l'application doit proposer des interfaces simples, intuitives et cohérentes afin de faciliter la navigation et l'utilisation des différentes fonctionnalités par les Customers, Employees et Admins.
- **Sécurité** : la solution doit garantir la confidentialité et l'intégrité des données des utilisateurs. L'accès aux ressources protégées doit être contrôlé à l'aide d'un mécanisme d'authentification sécurisé basé sur Spring Security et JWT. Les mots de passe doivent être stockés de manière sécurisée et les utilisateurs doivent uniquement accéder aux fonctionnalités qui leur sont autorisées.
- **Contrôle d'accès** : le système doit appliquer des droits d'accès en fonction des rôles attribués aux utilisateurs. Pour les employés, les autorisations doivent égale-

ment prendre en compte leur département afin de limiter l'accès aux fonctionnalités nécessaires à leurs responsabilités.

- **Performance** : l'application doit garantir des temps de réponse satisfaisants lors de l'utilisation des différentes fonctionnalités. Les traitements effectués par le système doivent être optimisés afin d'assurer une utilisation fluide de la plateforme.
- **Fiabilité** : le système doit garantir la cohérence et l'intégrité des données manipulées. Les opérations telles que les commandes, les paiements, les mouvements de stock et les opérations de maintenance doivent être enregistrées de manière fiable.
- **Disponibilité** : les fonctionnalités principales de la plateforme doivent être accessibles de manière stable afin de permettre aux utilisateurs de réaliser leurs opérations sans interruption anormale.
- **Maintenabilité** : le code source doit être organisé, structuré et compréhensible afin de faciliter la correction des erreurs, la maintenance du système et l'ajout de nouvelles fonctionnalités.
- **Modularité et extensibilité** : la solution doit être conçue de manière modulaire afin de permettre l'évolution indépendante des différents modules fonctionnels. Cette organisation doit faciliter l'intégration de nouvelles fonctionnalités sans modifier de manière importante les fonctionnalités existantes.
- **Évolutivité** : l'architecture doit permettre à la plateforme d'évoluer en fonction des futurs besoins de MSG, notamment par l'ajout de nouveaux modules, de nouveaux rôles ou de nouvelles fonctionnalités métier.
- **Compatibilité** : l'application doit être accessible depuis les principaux navigateurs web utilisés par les utilisateurs et garantir une expérience cohérente sur les différents appareils pris en charge.

1.6 Méthodologie de développement

Pour organiser le développement de la plateforme MSG, nous avons adopté une approche agile basée sur la méthodologie Scrum. Ce choix est lié à la nature du projet, qui regroupe plusieurs modules et dont certaines fonctionnalités peuvent évoluer selon les besoins du client.

Le principe de Scrum consiste à diviser le développement en plusieurs périodes courtes appelées *Sprints*. Chaque Sprint se concentre sur un ensemble de fonctionnalités prioritaires et aboutit à une version fonctionnelle d'une partie de la solution.

Cette organisation nous permet de suivre l'avancement du projet de manière progressive, de vérifier régulièrement les fonctionnalités développées et d'intégrer les éventuels retours du client.

1.6.1 Méthodologie Scrum

Scrum est un cadre de travail agile fondé sur un développement itératif et progressif. Au lieu de réaliser l'ensemble du projet en une seule étape, les fonctionnalités sont réparties sur plusieurs Sprints.

Pour chaque Sprint, l'équipe sélectionne les fonctionnalités les plus importantes à partir du *Product Backlog*. Ces fonctionnalités sont ensuite analysées, conçues, développées et testées avant la fin de l'itération.

Dans le cadre de notre projet, les Sprints sont organisés autour des principaux domaines fonctionnels de la plateforme MSG, notamment :

- la gestion des utilisateurs et de l'authentification ;
- la gestion des produits, du stock et des fournisseurs ;
- la gestion des ventes, commandes, factures et paiements ;
- la gestion des services de maintenance ;
- la gestion des employés et des ressources humaines ;
- les fonctionnalités e-commerce et les fonctionnalités intelligentes.

1.6.2 Les rôles au sein de l'équipe Scrum

L'organisation Scrum repose sur une répartition des responsabilités entre les différents membres participant au projet.

— **Product Owner :**

Le Product Owner est chargé de représenter les besoins du client et de maintenir une vision globale du produit. Il participe notamment à :

- l'identification des besoins fonctionnels ;
- la définition et la priorisation des User Stories ;
- la gestion du Product Backlog ;
- la validation des fonctionnalités réalisées.

Dans notre projet, ce rôle est assuré par **M. Ahmed Ben Salah**, représentant du client MSG.

— **Scrum Master :**

Le Scrum Master accompagne l'équipe dans l'application de Scrum. Il veille à la bonne organisation du travail et facilite la résolution des difficultés rencontrées pendant le développement.

Ses principales responsabilités sont :

- organiser et faciliter les réunions Scrum ;
- suivre le déroulement des Sprints ;
- identifier les obstacles pouvant ralentir l'équipe ;
- favoriser la communication entre les différents intervenants ;
- encourager l'amélioration continue.

Ce rôle est assuré par **M. Mohamed Trabelsi**, chef de projet chez Qartis Technologies.

— **Équipe de développement :**

L'équipe de développement prend en charge la réalisation technique de la plateforme. Elle intervient dans les différentes étapes allant de l'analyse jusqu'aux tests.

Dans le cadre de notre projet, l'équipe assure notamment :

- la conception des fonctionnalités ;
- le développement frontend ;
- le développement backend ;
- la conception et la gestion de la base de données ;
- l'intégration des différents composants ;
- les tests et la correction des anomalies.

1.6.3 Les éléments de gestion de Scrum

Pour organiser les fonctionnalités et suivre l'avancement du projet, plusieurs éléments de Scrum sont utilisés.

— **Product Backlog :**

Le Product Backlog représente la liste complète des fonctionnalités identifiées pour la plateforme MSG. Les besoins sont généralement décrits sous forme de User Stories et classés selon leur niveau de priorité.

— **Sprint Backlog :**

Le Sprint Backlog contient les User Stories sélectionnées pour un Sprint donné. Il permet à l'équipe de déterminer les fonctionnalités qui doivent être réalisées pendant l'itération ainsi que les tâches nécessaires à leur réalisation.

— **User Story :**

Une User Story décrit une fonctionnalité du système du point de vue de l'utilisateur. Elle permet d'exprimer clairement le besoin, l'utilisateur concerné et l'objectif recherché.

Dans notre projet, les User Stories concernent principalement les acteurs **Cus-tomer**, **Employee** et **Admin**.

1.6.4 Déroulement d'un Sprint

Chaque Sprint suit une succession d'étapes permettant de transformer les besoins sélectionnés en fonctionnalités opérationnelles.

— **Planification du Sprint :**

Au début du Sprint, l'équipe examine les éléments prioritaires du Product Backlog. Les User Stories retenues sont intégrées au Sprint Backlog en fonction de

leur priorité et de la capacité de l'équipe. Un objectif est également défini pour l'itération.

— **Analyse et conception :**

Les besoins sélectionnés sont étudiés afin de comprendre les règles métier et les interactions nécessaires. Les diagrammes UML appropriés sont ensuite élaborés afin de préparer l'implémentation.

— **Développement :**

Les fonctionnalités sont implémentées par l'équipe de développement. Cette phase concerne les interfaces Angular, les services backend, les API REST, la base de données ainsi que les différents mécanismes nécessaires au fonctionnement du système.

— **Suivi quotidien :**

L'équipe réalise des points réguliers afin de suivre l'avancement des tâches, signaler les difficultés et coordonner les travaux en cours.

— **Tests et validation :**

Les fonctionnalités développées sont testées afin de vérifier leur conformité avec les besoins et les critères d'acceptation définis. Les anomalies identifiées sont corrigées avant la validation du Sprint.

— **Sprint Review :**

À la fin du Sprint, les fonctionnalités réalisées sont présentées afin de vérifier les résultats obtenus et de recueillir les remarques nécessaires pour la suite du projet.

— **Sprint Retrospective :**

Une rétrospective permet à l'équipe d'analyser le déroulement du Sprint. Les points positifs, les difficultés et les améliorations possibles sont identifiés afin d'améliorer le travail durant les prochaines itérations.

1.6.5 Langage de modélisation UML

Après avoir défini la méthodologie de développement, nous avons choisi le langage de modélisation utilisé pour représenter les différents aspects de la solution.

Nous avons adopté **UML** (*Unified Modeling Language*), qui constitue un langage standard permettant de représenter graphiquement la structure et le comportement d'un système logiciel.

Dans le cadre de la plateforme MSG, UML est utilisé principalement pour les diagrammes suivants :

— **Diagramme de cas d'utilisation :**

Il permet d'identifier les acteurs du système et de représenter les fonctionnalités auxquelles ils peuvent accéder.

— **Diagramme de séquence :**

Il décrit le déroulement chronologique d'une fonctionnalité en représentant les échanges entre les acteurs, les interfaces et les différents composants du système.

— **Diagramme de classes :**

Il représente la structure statique de la solution. Il permet d'identifier les principales classes, leurs attributs, leurs méthodes ainsi que les relations entre elles.

L'utilisation d'UML permet ainsi de disposer d'une représentation claire du système avant son implémentation et facilite la communication entre les différents intervenants du projet.

1.7 Conclusion

Ce chapitre a permis de présenter le contexte général du projet, l'organisme d'accueil ainsi que le client MSG. L'étude des besoins et l'analyse de l'existant ont permis d'identifier les principales limites des solutions disponibles et de justifier le développement d'une plateforme personnalisée.

Les exigences fonctionnelles et non fonctionnelles, les acteurs du système ainsi que les choix méthodologiques et de modélisation ont également été définis. La solution proposée vise ainsi à intégrer les fonctionnalités e-commerce, ERP et maintenance au sein d'une plateforme unique, sécurisée et évolutive.

Le chapitre suivant sera consacré au pilotage du projet et à la conception générale de la solution.

Chapitre 2

Pilotage du projet et conception générale

2.1 Introduction

Ce chapitre présente l'organisation du développement de la solution **AutoParts Pro** ainsi que les principaux choix effectués avant sa réalisation. Nous commençons par présenter le *Product Backlog*, qui regroupe les fonctionnalités identifiées à partir des besoins de MSG, puis nous décrivons la répartition de ces fonctionnalités à travers les différents Sprints.

Nous présentons ensuite le diagramme de cas d'utilisation global afin de donner une vue d'ensemble des interactions entre les acteurs et la plateforme. La suite du chapitre est consacrée aux choix technologiques et aux outils utilisés durant le développement. Enfin, nous décrivons l'architecture générale de la solution et la répartition de ses différents composants.

2.2 Pilotage du projet avec Scrum

Afin d'organiser le développement de la plateforme **AutoParts Pro**, nous avons adopté une approche agile basée sur le cadre de travail Scrum. Ce choix permet de diviser le projet en plusieurs itérations et de développer progressivement les différentes fonctionnalités de la solution.

Le développement est ainsi organisé en plusieurs Sprints, chacun étant associé à un objectif fonctionnel précis. Cette organisation permet de suivre régulièrement l'avancement du projet, de tester les fonctionnalités développées et d'apporter des ajustements lorsque cela est nécessaire.

2.2.1 Product Backlog

Le *Product Backlog* constitue une liste structurée des besoins fonctionnels identifiés pour la plateforme. Ces besoins sont exprimés sous forme de *User Stories*, permettant de préciser l'acteur concerné, la fonctionnalité souhaitée et l'objectif recherché.

Les User Stories sont classées selon leur priorité et associées à une estimation de leur charge de réalisation. Le tableau 2.1 présente une synthèse du backlog retenu pour notre projet.

ID	Module	User Story	Priorité	Est.
US1	Authentification	En tant qu'utilisateur, je souhaite créer un compte et m'authentifier afin d'accéder aux fonctionnalités qui me sont autorisées.	Élevée	5j
US2	Autorisations	En tant qu'Admin, je souhaite gérer les rôles et les permissions afin de contrôler l'accès aux différentes fonctionnalités.	Élevée	5j
US3	Gestion des employés	En tant qu'Admin, je souhaite créer et gérer les comptes des employés ainsi que leurs départements.	Élevée	6j
US4	Produits	En tant qu'Employee, je souhaite gérer les produits, catégories, marques et informations associées.	Élevée	7j
US5	Stock	En tant qu'Employee, je souhaite consulter et gérer les stocks afin de suivre les mouvements des produits.	Élevée	7j
US6	Fournisseurs	En tant qu'Employee, je souhaite gérer les fournisseurs et les opérations d'approvisionnement.	Moyenne	5j
US7	E-commerce	En tant que Customer, je souhaite consulter et rechercher les pièces automobiles disponibles afin de trouver les produits adaptés à mes besoins.	Élevée	7j
US8	Commandes	En tant que Customer, je souhaite gérer mon panier et passer une commande afin d'acheter les produits sélectionnés.	Élevée	7j
US9	Paielements	En tant que Customer, je souhaite effectuer le paiement de ma commande et consulter les informations relatives à celui-ci.	Élevée	5j
US10	Maintenance	En tant que Customer, je souhaite réserver un service de maintenance en renseignant les informations relatives à mon véhicule.	Élevée	8j
US11	Interventions	En tant qu'Employee, je souhaite gérer les demandes de maintenance et suivre les interventions réalisées.	Élevée	7j
US12	Ressources humaines	En tant qu'Admin, je souhaite gérer les informations des employés, les congés, les horaires et les shifts.	Moyenne	8j
US13	Automatisation	En tant qu'administrateur, je souhaite automatiser certaines opérations afin de réduire les tâches répétitives.	Moyenne	6j

TABLE 2.1 – Product Backlog du projet AutoParts Pro

2.2.2 Planification des Sprints

Après l'identification des fonctionnalités, le Product Backlog a été réparti sur plusieurs Sprints en tenant compte des dépendances entre les modules et de leur priorité.

Le développement commence par les fonctionnalités nécessaires à la sécurisation de

la plateforme. Les modules de gestion des produits et des stocks sont ensuite développés avant d'intégrer les fonctionnalités liées à la maintenance, aux ressources humaines et à l'automatisation.

Le tableau 2.2 présente la planification générale retenue.

Sprint	Intitulé	Fonctionnalités principales	Période
Sprint 0	Initialisation du projet	Analyse des besoins, prise en main de l'environnement technique, définition de l'architecture et préparation du projet.	À préciser
Sprint 1	Authentification et autorisations	Authentification, inscription, vérification e-mail, réinitialisation du mot de passe, JWT, rôles et permissions.	À préciser
Sprint 2	Produits et gestion du stock	Gestion des produits, catégories, marques, fournisseurs et suivi des stocks.	À préciser
Sprint 3	ERP et services de maintenance	Gestion des opérations ERP, réservations de maintenance, véhicules, demandes et interventions.	À préciser
Sprint 4	Ressources humaines	Gestion des employés, départements, horaires, shifts et congés.	À préciser
Sprint 5	E-commerce et automatisation	Commandes, paiements, factures, fonctionnalités e-commerce et automatisation des processus.	À préciser

TABLE 2.2 – Planification des Sprints

Cette organisation permet de construire progressivement la solution en commençant par les fonctionnalités fondamentales, puis en ajoutant les différents modules métier. Les détails de chaque Sprint seront présentés dans les chapitres dédiés à leur réalisation.

2.3 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global permet de présenter les principales fonctionnalités de la plateforme ainsi que les interactions entre les différents acteurs.

Dans notre solution, trois utilisateurs principaux interagissent directement avec la plateforme : le **Customer**, l'**Employee** et l'**Admin**. Le système intervient également pour réaliser automatiquement certaines opérations, notamment l'envoi des e-mails et la gestion des mécanismes d'authentification.

Le **Customer** utilise principalement la partie e-commerce et les services de maintenance. Il peut créer son compte, consulter les produits, rechercher des pièces, gérer son panier, effectuer des commandes et réserver des interventions.

L'**Employee** intervient principalement dans la gestion opérationnelle de l'entreprise. Selon ses permissions, il peut gérer les produits, les stocks, les fournisseurs, les commandes ou encore les interventions de maintenance.

L'**Admin** dispose de fonctionnalités d'administration permettant notamment de gérer les employés, les départements, les rôles et les permissions.

Le diagramme permet ainsi d'avoir une vision globale du système avant d'étudier séparément les fonctionnalités de chaque Sprint.

2.4 Choix techniques

La réalisation d'AutoParts Pro nécessite plusieurs technologies permettant de couvrir les différents niveaux de la solution. Les choix techniques ont été effectués en tenant compte des besoins du projet, de la maintenabilité du code, de la sécurité et de la possibilité de faire évoluer la plateforme.

L'environnement technique est organisé autour de plusieurs éléments : le frontend destiné aux utilisateurs, le backend chargé de la logique métier, le système de gestion de données ainsi que les outils utilisés pour le développement et la collaboration.

2.4.1 Environnement logiciel

Technologies du frontend

Angular : est un framework open source développé par Google pour la conception d'applications web dynamiques. Il repose notamment sur TypeScript et propose une architecture basée sur les composants et les services.

Dans notre projet, Angular est utilisé pour développer les interfaces de l'application, notamment les interfaces d'administration de l'ERP et les différentes fonctionnalités accessibles aux utilisateurs.

TypeScript : est un langage développé par Microsoft qui étend JavaScript en ajoutant notamment le typage statique. Il facilite la structuration et la maintenance des applications web de grande taille.

TypeScript est utilisé dans notre projet pour développer les composants, services, modèles et différents éléments de l'application Angular.

HTML5 et CSS3 : sont utilisés pour structurer et mettre en forme les interfaces web. Ils permettent de construire des interfaces adaptées aux différentes fonctionnalités de la plateforme.

Tailwind CSS : est utilisé pour faciliter la conception des interfaces et permettre une mise en page cohérente et adaptable aux différentes tailles d'écran.

Technologies du backend

Java : constitue le langage principal utilisé pour le développement de la partie serveur. Sa robustesse, son typage et son large écosystème en font une technologie adaptée à la

réalisation d'applications métier.

Spring Boot : est un framework Java permettant de développer des applications backend de manière structurée et efficace. Il facilite notamment la création d'API REST et l'organisation de la logique métier.

Dans AutoParts Pro, Spring Boot assure le traitement des requêtes provenant du frontend, l'application des règles métier et l'accès aux données.

Spring Security : est utilisé pour mettre en place les mécanismes d'authentification et d'autorisation. Il permet notamment de sécuriser les ressources de l'application et de contrôler les accès selon les rôles et permissions.

JWT (*JSON Web Token*) : est utilisé pour transmettre de manière sécurisée les informations nécessaires à l'authentification entre le frontend et le backend. Après une authentification réussie, un token est généré puis utilisé lors de l'accès aux ressources protégées.

Système de gestion des données

MySQL : est un système de gestion de bases de données relationnelles. Il permet de stocker et d'organiser les informations nécessaires au fonctionnement de la plateforme.

Dans notre projet, la base de données contient notamment les informations relatives aux utilisateurs, employés, produits, catégories, fournisseurs, stocks, commandes, paiements, factures et opérations de maintenance.

Spring Data JPA : est utilisé pour faciliter l'interaction entre l'application Java et la base de données. Il permet de manipuler les données à travers des repositories et de réduire le code nécessaire aux opérations courantes d'accès aux données.

2.5 Outils de développement

Plusieurs outils ont été utilisés afin de faciliter la réalisation, le test et la maintenance de la plateforme.

2.5.1 Environnement de développement

Visual Studio Code : est un éditeur de code utilisé principalement pour le développement de la partie Angular. Il propose de nombreuses extensions facilitant le développement TypeScript, Angular, HTML et CSS.

IntelliJ IDEA : est un environnement de développement intégré utilisé pour la réalisation de la partie backend Java et Spring Boot. Il fournit notamment des outils d'analyse du code, de débogage et de gestion des projets Maven.

2.5.2 Outil de construction

Maven : est utilisé pour gérer la construction du projet backend ainsi que les dépendances nécessaires à l'application Spring Boot. Il permet également d'automatiser différentes opérations liées au cycle de développement.

2.5.3 Outils de gestion de versions

Git : est un système de gestion de versions distribué permettant de conserver l'historique des modifications apportées au code source et de travailler avec différentes branches.

GitHub : est utilisé comme plateforme d'hébergement des dépôts Git. Il facilite la sauvegarde du code, la gestion des branches et le suivi des différentes versions du projet.

2.5.4 Outil de gestion du projet

Trello : peut être utilisé pour organiser les tâches du projet sous forme de tableaux et de cartes. Il permet de suivre l'état d'avancement des différentes User Stories et tâches associées aux Sprints.

2.6 Outils de conception et de modélisation

PlantUML : est un outil permettant de générer différents types de diagrammes à partir d'une description textuelle. Il est utilisé dans notre projet pour représenter notamment les diagrammes UML.

Cette approche facilite la modification des diagrammes au cours de l'évolution du projet et permet de conserver une représentation cohérente avec la conception du système.

Les diagrammes utilisés dans le cadre du projet comprennent principalement :

- les diagrammes de cas d'utilisation ;
- les diagrammes de classes ;
- les diagrammes de séquence.

2.7 Architecture de l'application

Le choix de l'architecture représente une étape importante dans la conception d'AutoParts Pro. Une organisation claire des différentes responsabilités permet de faciliter la maintenance, l'évolution et la sécurisation de la plateforme.

Nous avons adopté une architecture en couches séparant principalement la présentation, la logique métier et l'accès aux données. Cette organisation permet de limiter les dépendances directes entre les différentes parties du système.

2.7.1 Couche présentation

La couche présentation correspond à l'application frontend développée avec Angular. Elle assure l'interaction avec les utilisateurs et prend en charge l'affichage des informations ainsi que la récupération des données depuis le backend.

Elle est principalement composée des éléments suivants :

- **Components** : ils représentent les différentes interfaces et assurent la gestion des interactions avec les utilisateurs.
- **Services Angular** : ils assurent la communication avec le backend à travers des requêtes HTTP et permettent de centraliser les appels aux différentes API.
- **Models** : ils permettent de représenter côté frontend les données manipulées par l'application.
- **Guards et Interceptors** : ils participent à la sécurisation de l'application frontend, notamment en contrôlant l'accès aux routes et en ajoutant les informations d'authentification aux requêtes.

2.7.2 Couche métier

La couche métier correspond à l'application backend développée avec Spring Boot. Elle reçoit les requêtes provenant du frontend, applique les règles métier et retourne les résultats nécessaires.

Elle est organisée autour de plusieurs composants :

- **Controllers** : ils reçoivent les requêtes HTTP et exposent les différentes API REST de l'application.
- **Services** : ils contiennent la logique métier et assurent le traitement des opérations demandées.
- **Repositories** : ils assurent l'accès aux données stockées dans la base de données.
- **Entities** : elles représentent les principales données métier manipulées par l'application et sont associées aux tables de la base de données.
- **DTO** : les objets de transfert de données permettent d'échanger uniquement les informations nécessaires entre le frontend et le backend.
- **Mappers** : ils assurent la conversion entre les entités métier et les DTO utilisés pour les échanges avec le frontend.

2.7.3 Couche données

La couche données correspond à la base de données relationnelle MySQL. Elle assure la persistance des informations utilisées par les différents modules de la plateforme.

Les données stockées concernent notamment :

- les utilisateurs et leurs rôles ;

- les employés et les départements ;
- les produits, catégories et marques ;
- les fournisseurs et les stocks ;
- les clients et les véhicules ;
- les commandes et les lignes de commande ;
- les paiements et les factures ;
- les services et les interventions de maintenance.

Cette séparation en différentes couches permet de mieux organiser le code et de faciliter les évolutions futures de la plateforme.

2.8 Architecture générale des échanges

Le fonctionnement général de la plateforme repose sur une communication entre le frontend Angular et le backend Spring Boot à travers des API REST.

Lorsqu'un utilisateur effectue une opération depuis l'interface, le composant Angular concerné transmet la demande au service correspondant. Celui-ci envoie une requête HTTP vers l'API REST du backend.

Le contrôleur Spring Boot reçoit ensuite la requête et la transmet au service métier approprié. Le service applique les règles nécessaires et utilise les repositories pour accéder aux données stockées dans MySQL.

Une fois le traitement terminé, le résultat est retourné au frontend sous forme de données JSON afin d'être affiché à l'utilisateur.

Cette organisation permet de séparer clairement l'interface utilisateur, la logique métier et la persistance des données, tout en facilitant la maintenance et l'évolution de la solution.

2.9 Conclusion

Dans ce chapitre, nous avons présenté l'organisation du projet AutoParts Pro à travers le Product Backlog et la planification des différents Sprints. Nous avons également décrit le diagramme de cas d'utilisation global afin de présenter les principales interactions entre les acteurs et le système.

Nous avons ensuite présenté les technologies et les outils retenus pour le développement, notamment Angular, TypeScript, Java, Spring Boot, Spring Security, JWT et MySQL, ainsi que les outils utilisés pour le développement, la gestion des versions et la modélisation.

Enfin, l'architecture générale de la plateforme a été décrite en présentant les responsabilités des différentes couches et les échanges entre le frontend, le backend et la base de données.

Ces choix constituent la base technique et organisationnelle nécessaire à la réalisation progressive de la solution. Les chapitres suivants seront consacrés à la réalisation des différents Sprints et à la présentation détaillée des fonctionnalités développées.

Chapitre 3

Réalisation du Sprint 1 : Gestion des utilisateurs et Authentification

3.1 Introduction

Ce premier sprint a pour objectif de mettre en place le système de gestion des utilisateurs ainsi que les mécanismes de sécurisation de l'application MSG. Il constitue la base de la plateforme en assurant l'authentification des différents acteurs, la protection des ressources et le contrôle des accès selon les rôles attribués.

La solution développée repose sur **Spring Security** et l'authentification par **JSON Web Token (JWT)** afin de garantir un accès sécurisé aux différentes fonctionnalités de l'application. Ce sprint couvre également l'inscription des clients, la vérification de l'adresse e-mail, la gestion des mots de passe, la création des employés par l'administrateur ainsi que la gestion des autorisations en fonction des rôles et des départements.

3.2 Analyse du Sprint

Cette section présente l'analyse du premier sprint consacré à la gestion des utilisateurs et à la sécurisation de l'application MSG. Elle décrit les fonctionnalités retenues et les principaux cas d'utilisation réalisés pour répondre aux besoins des différents acteurs du système.

3.2.1 Sprint Backlog

Le Sprint Backlog présente les fonctionnalités sélectionnées sous forme de User Stories, en fonction de leur priorité et de leur importance pour la mise en place de l'authentification, de la gestion des utilisateurs et du contrôle des accès selon les rôles.

ID	User Story	Critère d'acceptation	Priorité
US1	En tant que système, je souhaite initialiser automatiquement un compte administrateur principal au premier démarrage afin de garantir l'accès initial à la plateforme MSG.	Le compte Admin est créé automatiquement avec un rôle administrateur et les permissions nécessaires. La création est effectuée uniquement si aucun compte administrateur n'existe dans la base de données.	Haute
US2	En tant que Customer, je souhaite créer un compte afin d'accéder aux services de l'application MSG.	Le compte est créé et un e-mail de vérification est envoyé.	Haute
US3	En tant que Customer, je souhaite vérifier mon adresse e-mail afin d'activer mon compte.	Le compte est activé après validation du lien reçu par e-mail.	Moyenne
US4	En tant que Customer, je souhaite me connecter afin d'accéder aux fonctionnalités qui me sont autorisées.	Une authentification réussie génère un jeton JWT.	Haute
US5	En tant qu'utilisateur, je souhaite réinitialiser mon mot de passe lorsque je l'ai oublié.	Un e-mail contenant un lien de réinitialisation est envoyé.	Moyenne
US6	En tant qu'Admin, je souhaite me connecter à la plateforme afin d'administrer le système.	L'administrateur est authentifié et accède au tableau de bord d'administration.	Haute
US7	En tant qu'Admin, je souhaite créer un compte employé afin de lui donner accès à la plateforme.	Le compte employé est créé avec un rôle et un département puis les identifiants sont envoyés par e-mail.	Haute
US8	En tant qu'Employee, je souhaite définir un nouveau mot de passe lors de ma première connexion afin de sécuriser mon compte.	Le mot de passe est modifié et le compte est activé.	Moyenne

ID	User Story	Critère d'acceptation	Priorité
US9	En tant qu'Employee, je souhaite me connecter afin d'accéder aux fonctionnalités correspondant à mon département.	L'authentification réussie génère un JWT contenant les informations du rôle et du département.	Haute
US10	En tant qu'Administrateur, je souhaite attribuer un rôle et un département à chaque employé afin de contrôler les autorisations d'accès.	Les permissions sont appliquées conformément au rôle et au département attribués.	Moyenne
US11	En tant qu'utilisateur authentifié, je souhaite que mes accès soient contrôlés selon mon rôle afin de protéger les ressources de l'application.	Les accès sont autorisés ou refusés par Spring Security en fonction des rôles et permissions.	Haute

TABLE 3.1 – Sprint Backlog - Gestion des utilisateurs et sécurité

3.3 Conception

Cette section présente la conception détaillée des fonctionnalités développées durant le premier sprint. Elle décrit les interactions entre les différents composants du système ainsi que la structure des principales entités utilisées pour la gestion des utilisateurs et la sécurisation de l'application MSG.

3.3.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation présente les fonctionnalités liées à la gestion des utilisateurs et à la sécurisation de l'application MSG. Il décrit les interactions entre les différents acteurs du système et les services d'authentification disponibles.

Le système implique trois acteurs principaux :

- **Customer** : peut créer un compte, vérifier son adresse e-mail, se connecter et réinitialiser son mot de passe.
- **Admin** : peut se connecter, réinitialiser son mot de passe, créer des comptes Employee et gérer leurs rôles et permissions.
- **Employee** : reçoit ses informations de connexion après sa création, définit son mot

de passe, se connecte et utilise les fonctionnalités autorisées selon son rôle.

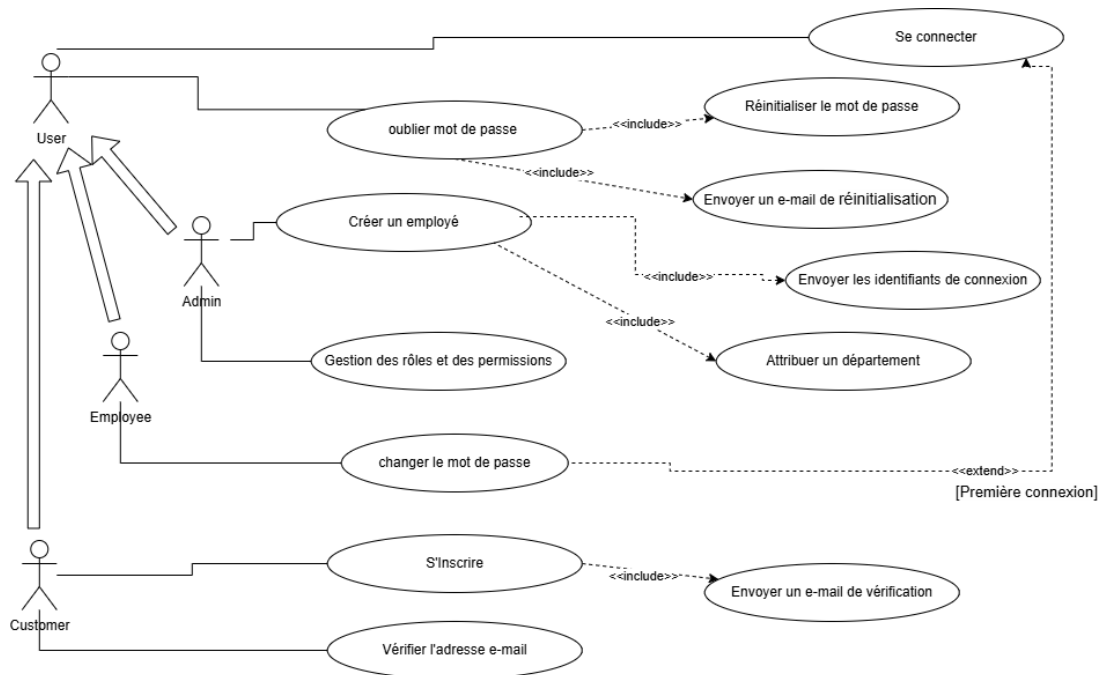


FIGURE 3.1 – Diagramme de cas d'utilisation global du Sprint 1

3.4 Description des Cas d'Utilisation

Afin de compléter le diagramme de cas d'utilisation présenté précédemment, les principaux cas d'utilisation du système sont décrits dans les tableaux suivants.

UC1 : Se connecter

Élément	Description
Cas d'utilisation	Se connecter
Acteur(s)	User, Customer, Employee, Admin
Préconditions	L'utilisateur possède un compte et dispose de ses identifiants de connexion.
Scénario	1. L'utilisateur accède à la page de connexion. 2. Il saisit son adresse e-mail et son mot de passe. 3. Le système vérifie les informations d'authentification. 4. Le système authentifie l'utilisateur. 5. Le système génère un jeton JWT. 6. L'utilisateur accède aux fonctionnalités correspondant à son rôle.
Postconditions	L'utilisateur est authentifié et peut accéder aux ressources autorisées selon son rôle et ses permissions.
Exceptions	Les identifiants sont incorrects, le compte est inactif ou l'utilisateur ne dispose pas des permissions nécessaires.

TABLE 3.2 – Description du cas d'utilisation « Se connecter »

UC2 : Oublier le mot de passe

Élément	Description
Cas d'utilisation	Oublier le mot de passe
Acteur(s)	User
Préconditions	L'utilisateur possède un compte enregistré dans le système.
Scénario	1. L'utilisateur sélectionne l'option « Mot de passe oublié ». 2. Il saisit son adresse e-mail. 3. Le système vérifie l'existence du compte. 4. Le système lance la procédure de réinitialisation du mot de passe.
Postconditions	La procédure de réinitialisation du mot de passe est déclenchée.
Exceptions	L'adresse e-mail n'est associée à aucun compte.

TABLE 3.3 – Description du cas d'utilisation « Oublier le mot de passe »

UC3 : Réinitialiser le mot de passe

Élément	Description
Cas d'utilisation	Réinitialiser le mot de passe
Acteur(s)	User
Préconditions	L'utilisateur a demandé une réinitialisation et dispose d'un lien ou d'un jeton de réinitialisation valide.
Scénario	1. L'utilisateur accède au lien de réinitialisation reçu par e-mail. 2. Le système vérifie la validité du jeton. 3. L'utilisateur saisit son nouveau mot de passe. 4. Le système valide le nouveau mot de passe. 5. Le système enregistre le nouveau mot de passe.
Postconditions	Le mot de passe est modifié et peut être utilisé pour une nouvelle connexion.
Exceptions	Le jeton est invalide ou expiré, ou le nouveau mot de passe ne respecte pas les règles de sécurité.

TABLE 3.4 – Description du cas d'utilisation « Réinitialiser le mot de passe »

UC4 : Envoyer un e-mail de réinitialisation

Élément	Description
Cas d'utilisation	Envoyer un e-mail de réinitialisation
Acteur(s)	Système
Préconditions	Une demande de réinitialisation a été effectuée et l'adresse e-mail correspond à un compte existant.
Scénario	1. Le système génère un jeton de réinitialisation. 2. Le système construit le lien de réinitialisation. 3. Le système envoie un e-mail contenant le lien à l'utilisateur.
Postconditions	L'utilisateur reçoit un e-mail contenant le lien de réinitialisation.
Exceptions	Une erreur peut survenir lors de la génération du jeton ou de l'envoi de l'e-mail.

TABLE 3.5 – Description du cas d'utilisation « Envoyer un e-mail de réinitialisation »

UC5 : Créer un employé

Élément	Description
Cas d'utilisation	Créer un employé
Acteur(s)	Admin
Préconditions	L'administrateur est authentifié et dispose des droits nécessaires pour créer un employé.
Scénario	1. L'administrateur accède à l'interface de création d'un employé. 2. Il saisit les informations de l'employé. 3. Le système vérifie les données saisies. 4. Le système crée le compte de l'employé. 5. Le système attribue un département à l'employé. 6. Le système envoie les identifiants de connexion par e-mail.
Postconditions	Le compte de l'employé est créé, son département est attribué et ses identifiants lui sont envoyés.
Exceptions	Les données saisies sont invalides ou l'adresse e-mail est déjà utilisée.

TABLE 3.6 – Description du cas d'utilisation « Créer un employé »

UC6 : Envoyer les identifiants de connexion

Élément	Description
Cas d'utilisation	Envoyer les identifiants de connexion
Acteur(s)	Système
Préconditions	Un compte employé vient d'être créé et dispose d'une adresse e-mail valide.
Scénario	1. Le système récupère les informations de connexion. 2. Le système prépare l'e-mail de bienvenue. 3. Le système envoie les informations de connexion à l'employé.
Postconditions	L'employé reçoit ses informations de connexion par e-mail.
Exceptions	Une erreur d'envoi de l'e-mail peut survenir.

TABLE 3.7 – Description du cas d'utilisation « Envoyer les identifiants de connexion »

UC7 : Attribuer un département

Élément	Description
Cas d'utilisation	Attribuer un département
Acteur(s)	Admin
Préconditions	L'employé existe dans le système et l'administrateur dispose des droits nécessaires.
Scénario	1. L'administrateur sélectionne un employé. 2. Il sélectionne le département à attribuer. 3. Le système vérifie l'existence de l'employé et du département. 4. Le système enregistre l'affectation.
Postconditions	L'employé est associé au département sélectionné.
Exceptions	L'employé ou le département n'existe pas, ou l'administrateur ne dispose pas des droits nécessaires.

TABLE 3.8 – Description du cas d'utilisation « Attribuer un département »

UC8 : Gestion des rôles et des permissions

Élément	Description
Cas d'utilisation	Gestion des rôles et des permissions
Acteur(s)	Admin
Préconditions	L'administrateur est authentifié et dispose des droits d'administration.
Scénario	1. L'administrateur accède à l'interface de gestion des rôles et permissions. 2. Il sélectionne un employé. 3. Il attribue ou modifie le rôle et les permissions. 4. Le système vérifie les données. 5. Le système enregistre les modifications.
Postconditions	Les rôles et les permissions de l'employé sont mis à jour.
Exceptions	L'administrateur ne dispose pas des droits nécessaires ou les données saisies sont invalides.

TABLE 3.9 – Description du cas d'utilisation « Gestion des rôles et des permissions »

UC9 : Changer le mot de passe

Élément	Description
Cas d'utilisation	Changer le mot de passe
Acteur(s)	Employee, User
Préconditions	L'utilisateur est authentifié ou effectue sa première connexion avec les identifiants reçus.
Scénario	1. L'utilisateur accède à l'interface de changement du mot de passe. 2. Il saisit son nouveau mot de passe. 3. Le système vérifie le respect des règles de sécurité. 4. Le système enregistre le nouveau mot de passe.
Postconditions	Le nouveau mot de passe est enregistré et peut être utilisé pour les prochaines connexions.
Exceptions	Le nouveau mot de passe ne respecte pas les règles de sécurité ou une erreur survient lors de l'enregistrement.

TABLE 3.10 – Description du cas d'utilisation « Changer le mot de passe »

UC10 : S'inscrire

Élément	Description
Cas d'utilisation	S'inscrire
Acteur(s)	Customer
Préconditions	Le client ne possède pas encore de compte et le formulaire d'inscription est accessible.
Scénario	1. Le client accède au formulaire d'inscription. 2. Il saisit ses informations personnelles. 3. Le système valide les informations saisies. 4. Le système crée le compte client. 5. Le système envoie un e-mail de vérification.
Postconditions	Le compte client est créé et reste inactif jusqu'à la vérification de l'adresse e-mail.
Exceptions	L'adresse e-mail est déjà utilisée ou les informations saisies sont invalides.

TABLE 3.11 – Description du cas d'utilisation « S'inscrire »

UC11 : Envoyer un e-mail de vérification

Élément	Description
Cas d'utilisation	Envoyer un e-mail de vérification
Acteur(s)	Système
Préconditions	Un compte client vient d'être créé avec une adresse e-mail valide.
Scénario	1. Le système génère un jeton de vérification. 2. Le système construit le lien de vérification. 3. Le système envoie l'e-mail de vérification au client.
Postconditions	Le client reçoit un e-mail contenant le lien de vérification de son adresse e-mail.
Exceptions	Une erreur peut survenir lors de la génération du jeton ou de l'envoi de l'e-mail.

TABLE 3.12 – Description du cas d'utilisation « Envoyer un e-mail de vérification »

UC12 : Vérifier l'adresse e-mail

Élément	Description
Cas d'utilisation	Vérifier l'adresse e-mail
Acteur(s)	Customer
Préconditions	Le compte client existe et un e-mail de vérification a été envoyé.
Scénario	1. Le client ouvre le lien de vérification reçu par e-mail. 2. Le système vérifie la validité du jeton. 3. Le système valide l'adresse e-mail. 4. Le système active le compte client.
Postconditions	L'adresse e-mail est vérifiée et le client peut se connecter à son compte.
Exceptions	Le jeton de vérification est invalide ou expiré.

TABLE 3.13 – Description du cas d'utilisation « Vérifier l'adresse e-mail »

3.4.1 Diagrammes de Séquence

Les diagrammes de séquence suivants illustrent le déroulement de deux fonctionnalités principales liées à la gestion des employés : la création d'un employé par l'administrateur et sa première authentification.

Diagramme de Séquence : Création d'un Employé

Le diagramme suivant décrit le processus de création d'un employé. L'administrateur saisit les informations nécessaires depuis l'interface Angular. La requête est ensuite transmise au backend Spring Boot, où les droits d'accès et les données sont vérifiés. Le système crée l'employé, son contrat ainsi que son solde de congés, puis envoie ses identifiants de connexion par e-mail.

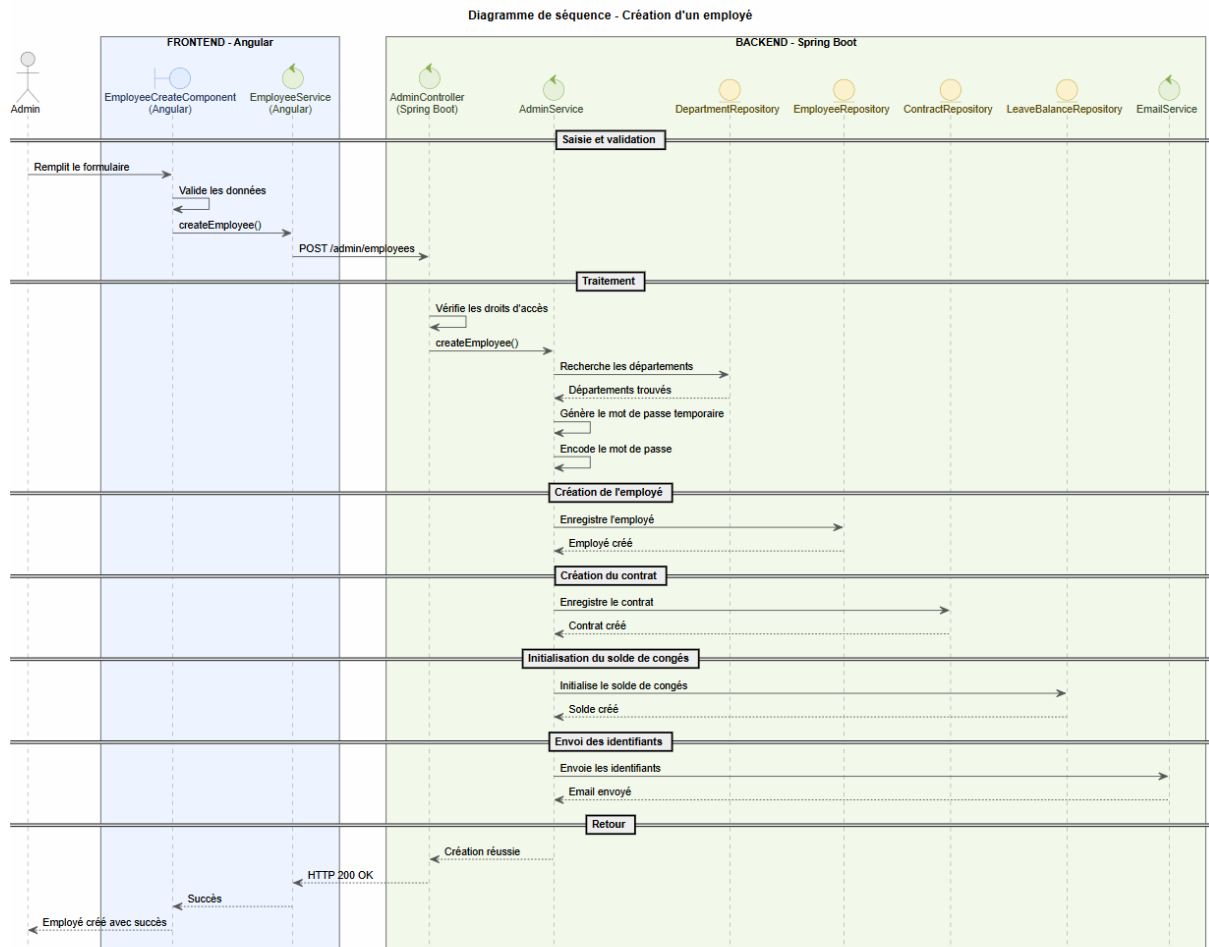


FIGURE 3.2 – Diagramme de séquence - Création d'un employé

Diagramme de Séquence : Première Authentification de l'Employé

Le diagramme suivant présente le processus d'authentification d'un employé. Après la saisie de ses identifiants, le système vérifie les informations d'authentification et génère un jeton JWT. Lors de la première connexion, l'employé est invité à modifier son mot de passe avant d'accéder au tableau de bord. En cas d'échec, le système retourne une erreur d'authentification.

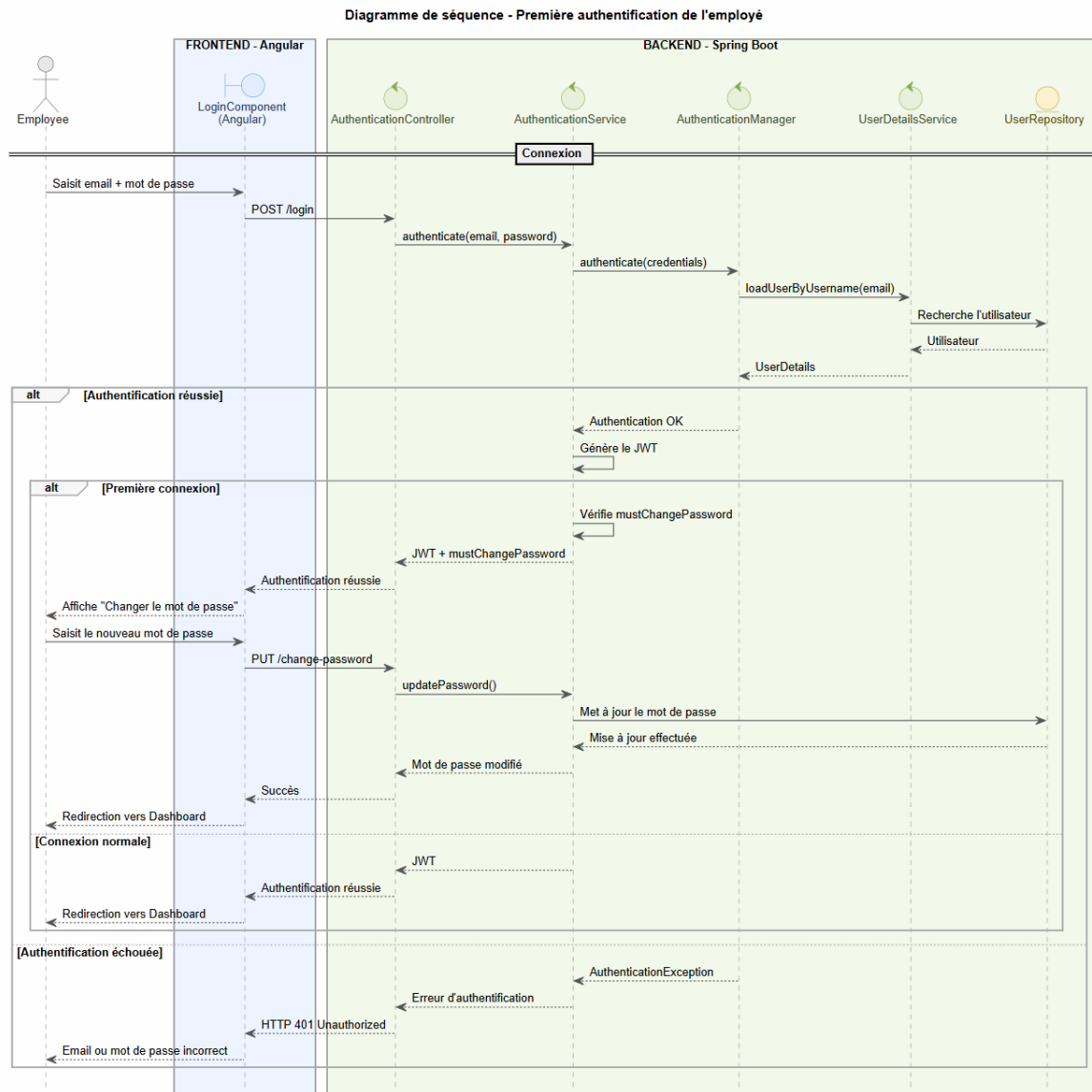


FIGURE 3.3 – Diagramme de séquence - Première authentification de l'employé

3.4.2 Diagramme de Classes

Le diagramme de classes présente la structure statique du module de gestion des utilisateurs. Il décrit les principales classes, leurs attributs ainsi que les relations qui existent entre elles.

Les principales classes représentées sont :

- **User** : classe principale regroupant les informations communes à tous les utilisateurs, telles que l'identité, les informations de contact et les données liées à l'authentification.
- **Customer** : représente les clients de la plateforme et contient notamment leurs informations de fidélité.
- **Employee** : représente les employés du système et contient leurs informations

professionnelles, leur disponibilité et leur statut.

- **Admin** : représente les administrateurs responsables de la gestion et de l'administration du système.
- **Department** : représente les différents départements auxquels les employés peuvent être affectés.
- **Permission** : représente les permissions disponibles dans le système et permet de définir les droits associés aux départements.
- **EmploymentStatus** : énumération permettant de représenter l'état d'un employé : *ACTIVE*, *ON_LEAVE*, *RESIGNED* ou *TERMINATED*.

Les principales relations sont les suivantes :

- **Customer**, **Employee** et **Admin** héritent de la classe **User** et partagent ainsi les informations communes liées aux utilisateurs.
- Un **Employee** est associé à un **Department**.
- Un **Department** peut être associé à plusieurs **Permission**.
- Le statut d'un **Employee** est défini à l'aide de l'énumération **EmploymentStatus**.

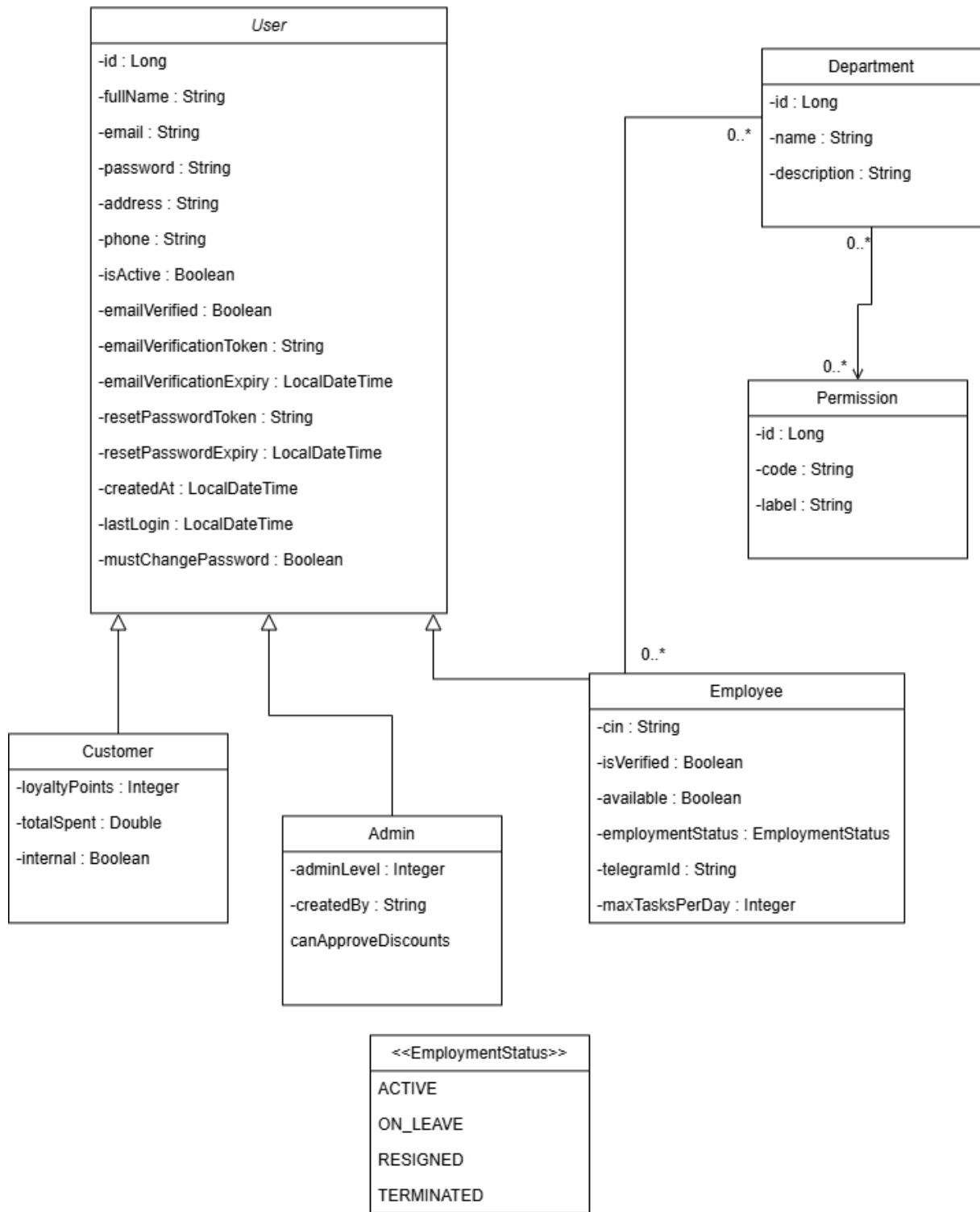


FIGURE 3.4 – Diagramme de classes - Gestion des utilisateurs

3.5 Architecture de Sécurité

La sécurité de l'application Management System Garage (MSG) repose sur l'intégration de *Spring Security* et du mécanisme d'authentification basé sur le JSON Web Token (JWT) (*JSON Web Token*). Cette architecture permet d'assurer l'authentification des

utilisateurs, la protection des ressources exposées par les API REST ainsi que le contrôle des accès en fonction des rôles attribués à chaque utilisateur.

Comme illustré par la Figure 3.5, le processus de sécurité débute lors de la connexion de l'utilisateur. Après vérification des informations d'authentification, un jeton JWT est généré puis transmis au client. Ce jeton est ensuite utilisé pour accéder aux ressources protégées, où il est systématiquement vérifié avant l'autorisation de chaque requête.

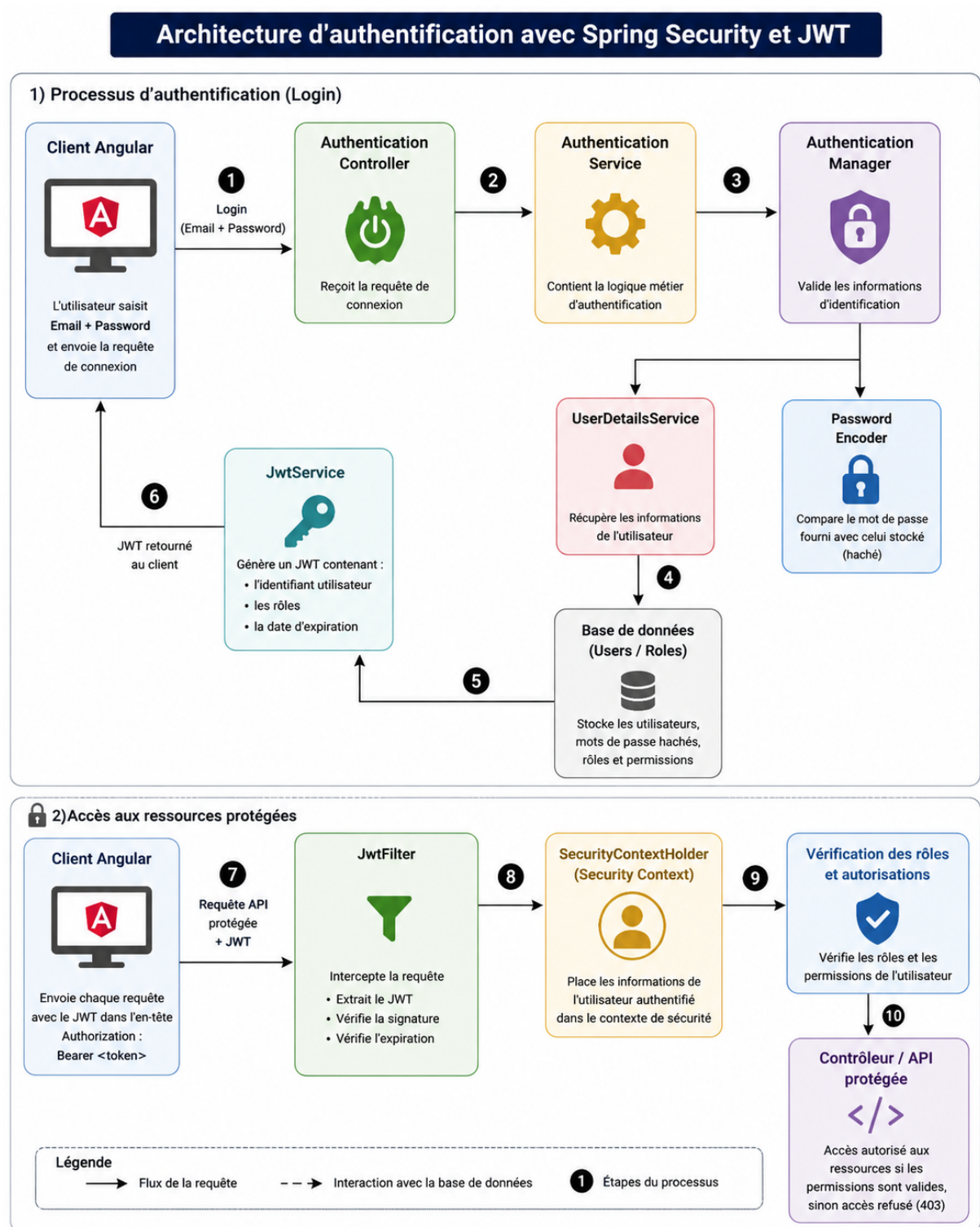


FIGURE 3.5 – Architecture de sécurité basée sur Spring Security et JWT

3.5.1 Architecture Spring Security

Spring Security est un framework dédié à la sécurisation des applications Spring. Dans l'application MSG, il prend en charge l'authentification des utilisateurs, la protection des API et le contrôle des autorisations en fonction des rôles.

Son fonctionnement repose principalement sur les composants suivants.

Security Filter Chain

La *Security Filter Chain* intercepte chaque requête HTTP avant son traitement par l'application. Elle vérifie la présence et la validité du JWT, identifie l'utilisateur authentifié et autorise ou refuse l'accès aux ressources protégées.

Authentication Provider

L'*Authentication Provider* est chargé de valider les informations de connexion. Il recherche l'utilisateur dans la base de données, compare le mot de passe fourni avec celui enregistré et confirme l'identité de l'utilisateur.

UserDetailsService

Le composant *UserDetailsService* récupère les informations nécessaires à l'authentification depuis la base de données, notamment l'identifiant, le mot de passe chiffré ainsi que les rôles associés à l'utilisateur.

PasswordEncoder

Le *PasswordEncoder* assure le chiffrement des mots de passe avant leur stockage et permet une comparaison sécurisée lors de l'authentification, garantissant ainsi qu'aucun mot de passe n'est conservé en clair.

Les mécanismes de sécurité présentés s'appuient sur les recommandations de Spring Security [1, 2].

3.5.2 Authentification basée sur JWT

Le JWT (*JSON Web Token*) est un standard permettant de transmettre de manière sécurisée les informations d'authentification entre le client et le serveur. Contrairement à une authentification basée sur les sessions, cette approche est *stateless* : le serveur ne conserve pas l'état de connexion des utilisateurs, les informations nécessaires étant contenues dans le jeton.

Le processus d'authentification se déroule selon les étapes suivantes.

1. L'utilisateur saisit son adresse e-mail et son mot de passe, qui sont envoyés au serveur.
2. Les identifiants sont vérifiés à l'aide du *UserDetailsService* et du *PasswordEncoder*. Si les informations sont valides, l'utilisateur est authentifié.

3. Le service *JwtService* génère un jeton JWT contenant l'identifiant de l'utilisateur, son rôle ainsi que sa date d'expiration.
4. Lors de chaque requête vers une ressource protégée, le client transmet ce jeton dans l'en-tête HTTP **Authorization**. Le filtre JWT valide alors le jeton avant de permettre l'accès à la ressource demandée.

Cette approche améliore les performances des applications distribuées en supprimant la gestion des sessions côté serveur tout en garantissant un niveau élevé de sécurité des échanges [3].

3.6 Réalisation

Cette section présente les principales fonctionnalités développées au cours de ce sprint concernant la gestion des utilisateurs et la sécurisation de l'application MSG. Elle décrit les différentes interfaces réalisées pour l'inscription des clients, l'authentification des utilisateurs, la réinitialisation des mots de passe ainsi que la création et la gestion des comptes des employés.

3.6.1 Inscription d'un Customer et vérification du compte par e-mail

Le processus d'inscription permet à un **Customer** de créer un compte dans l'application en renseignant les informations demandées dans le formulaire d'inscription. Après validation des données, le système enregistre le nouvel utilisateur puis envoie automatiquement un e-mail de confirmation contenant un lien de vérification. Le compte n'est activé qu'après la validation de cette adresse électronique.

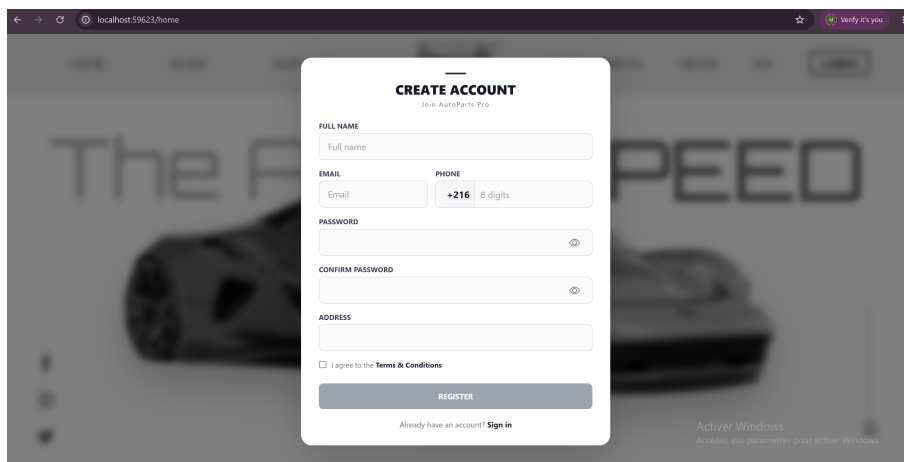


FIGURE 3.6 – Interface d'inscription d'un Customer

3.6.2 Authentification des utilisateurs

L'interface de connexion permet aux différents utilisateurs de l'application (**Customer**, **Admin** et **Employee**) d'accéder à leurs espaces respectifs. Après la saisie des identifiants, Spring Security vérifie les informations de connexion puis génère un jeton JWT permettant d'accéder aux ressources protégées. Selon le rôle de l'utilisateur, celui-ci est automatiquement redirigé vers son interface dédiée.

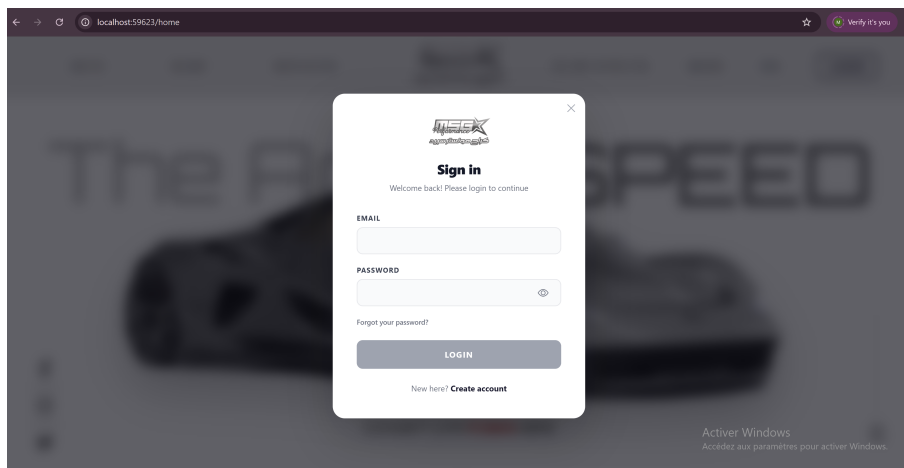


FIGURE 3.7 – Interface d'authentification Client

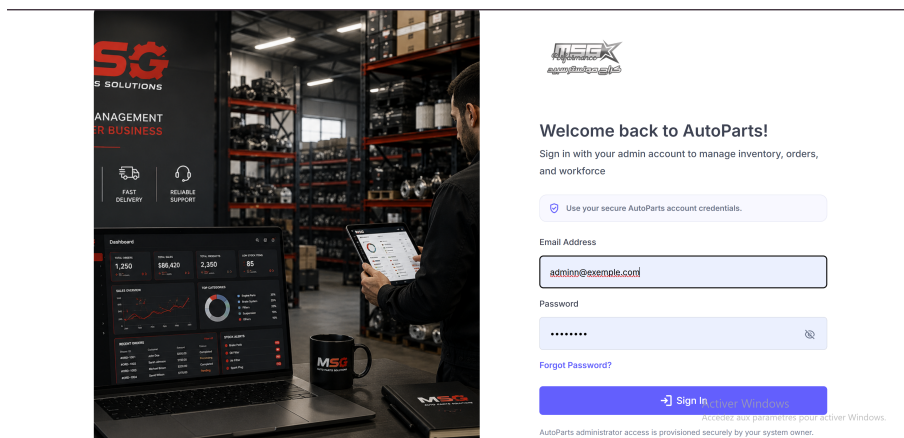


FIGURE 3.8 – Interface d'authentification Admin et employee

3.6.3 Réinitialisation du mot de passe

L'application offre une fonctionnalité permettant aux utilisateurs de récupérer l'accès à leur compte lorsqu'ils oublient leur mot de passe. L'utilisateur renseigne son adresse e-mail afin de recevoir un lien sécurisé de réinitialisation. Ce lien permet ensuite de définir un nouveau mot de passe.

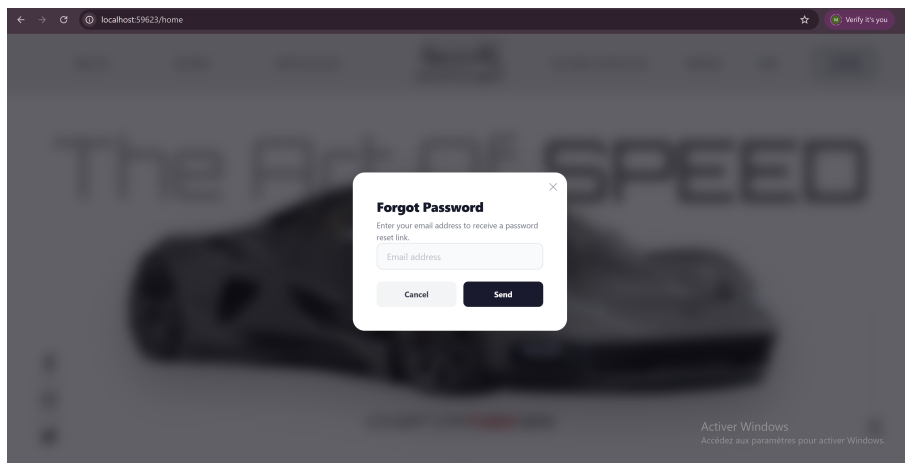


FIGURE 3.9 – Interface « Forgot Password «client» »

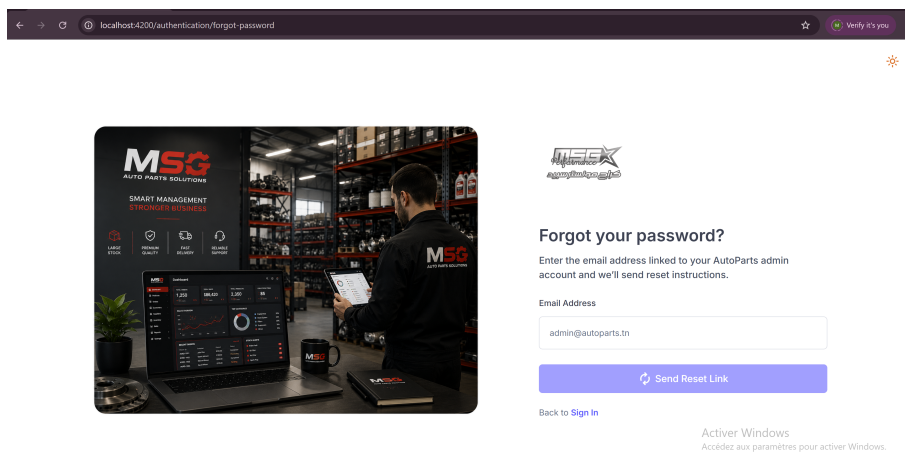


FIGURE 3.10 – Interface « Forgot Password «admin et employee» »

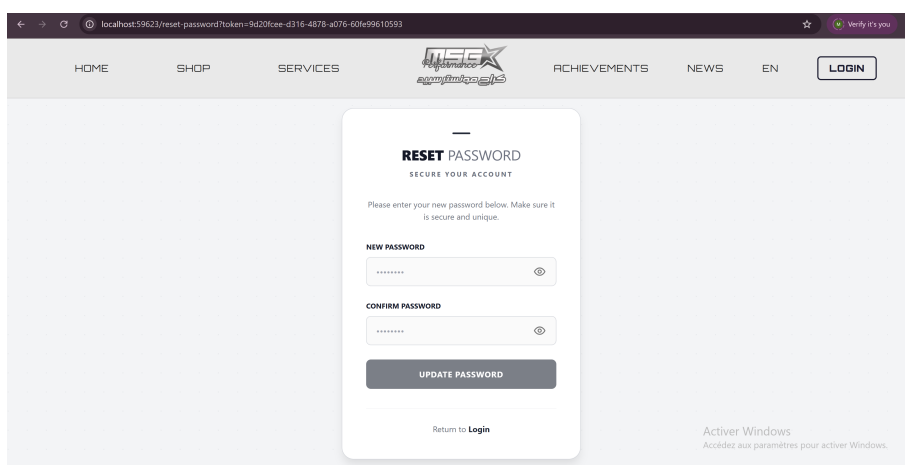


FIGURE 3.11 – Interface de réinitialisation du mot de passe client

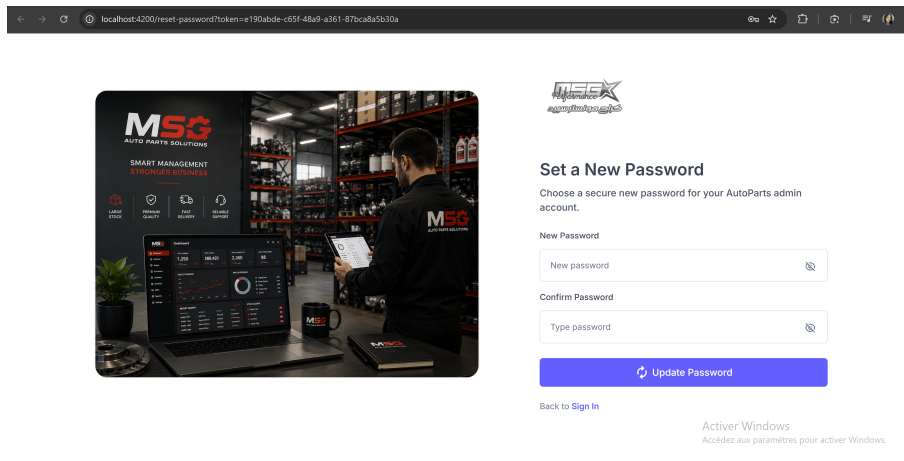


FIGURE 3.12 – Interface de réinitialisation du mot de passe admin et employee

3.6.4 Création et initialisation d'un Employee

Contrairement au **Customer**, un **Employee** ne crée pas son compte lui-même. Celui-ci est créé par l'**Admin** à partir de l'interface de gestion des employés. Lors de cette création, les informations de connexion sont automatiquement envoyées par courrier électronique. À sa première connexion, l'employé utilise les identifiants reçus afin d'accéder à son espace de travail.

FIGURE 3.13 – Interface de création d'un Employee

3.7 Bilan du Sprint

Ce premier sprint a permis de mettre en place les fondations de la gestion des utilisateurs et de la sécurité de l'application MSG. Les principales fonctionnalités d'authentification

tion, d'inscription, de vérification des comptes, de gestion des mots de passe et de création des employés ont été réalisées. La mise en place de Spring Security et du JWT permet également de sécuriser l'accès aux ressources et d'adapter les autorisations aux différents profils et départements. Ce socle constitue ainsi une base fiable pour le développement des fonctionnalités métiers des prochains sprints.

Chapitre 4

Réalisation du Sprint 2 : Gestion des produits et du stock

4.1 Introduction

Ce deuxième sprint est consacré à la gestion des produits et du stock. Il permet de mettre en place les fonctionnalités liées à la gestion du catalogue, des catégories, des marques, des fournisseurs et du stock.

Il couvre également les fonctionnalités de panier, de commandes, de facturation et de paiement, afin d'assurer la continuité entre la gestion des produits et les opérations de vente.

4.2 Analyse du Sprint

4.2.1 Sprint Backlog

Le Sprint Backlog regroupe les principales fonctionnalités à réaliser au cours de ce sprint. Il permet de définir les besoins fonctionnels liés à la gestion des produits, du stock, des fournisseurs ainsi qu'aux opérations de commande et de paiement.

ID	User Story	Critère d'acceptation	Priorité
US1	En tant qu'employé, je souhaite consulter la liste des produits afin de visualiser les produits disponibles.	La liste des produits est affichée avec leurs principales informations.	Haute

ID	User Story	Critère d'acceptation	Priorité
US2	En tant qu'employé, je souhaite rechercher, filtrer et trier les produits afin de retrouver rapidement un produit.	Les produits peuvent être recherchés, filtrés et triés selon les critères disponibles.	Haute
US3	En tant qu'employé, je souhaite ajouter, modifier et supprimer un produit afin de maintenir le catalogue à jour.	Un produit peut être créé, modifié ou supprimé avec succès.	Haute
US4	En tant qu'employé, je souhaite gérer les catégories et les marques afin d'organiser les produits.	Les catégories et les marques peuvent être ajoutées, modifiées ou supprimées selon les fonctionnalités disponibles.	Haute
US5	En tant qu'employé, je souhaite consulter et gérer le stock afin de suivre les quantités disponibles.	Les quantités et les informations du stock peuvent être consultées et mises à jour.	Haute
US6	En tant qu'employé, je souhaite enregistrer une réception de livraison afin de mettre à jour le stock.	La réception augmente la quantité disponible et enregistre le mouvement correspondant.	Haute
US7	En tant qu'employé, je souhaite consulter les mouvements de stock afin de suivre les entrées et les sorties.	Les mouvements sont affichés avec leurs informations principales et leur date.	Moyenne
US8	En tant qu'employé, je souhaite identifier les produits en stock faible afin d'anticiper leur réapprovisionnement.	Les produits dont la quantité est inférieure ou égale au seuil sont identifiés.	Haute

ID	User Story	Critère d'acceptation	Priorité
US9	En tant qu'employé, je souhaite gérer les fournisseurs afin de maintenir les informations d'approvisionnement à jour.	Un fournisseur peut être consulté, ajouté, modifié ou supprimé.	Moyenne
US10	En tant que client, je souhaite gérer mon panier afin de préparer ma commande.	Les produits peuvent être ajoutés, modifiés, supprimés ou retirés du panier et le total est calculé.	Haute
US11	En tant que client, je souhaite créer une commande et consulter sa facture afin de suivre mon achat.	La commande est enregistrée et les informations de facturation sont disponibles.	Haute
US12	En tant que client, je souhaite effectuer un paiement afin de confirmer ma commande.	Le paiement est enregistré, la commande est confirmée et le stock est mis à jour.	Haute

TABLE 4.1 – Sprint Backlog du Sprint 2

4.3 Conception

4.3.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation du Sprint 2 présente les interactions entre les différents acteurs du système et les principales fonctionnalités développées pour la gestion des produits, du stock, des fournisseurs ainsi que pour les opérations commerciales liées au panier, aux commandes et au paiement.

Dans ce sprint, quatre acteurs sont considérés : User, Customer, Employee et Admin. L'acteur User représente l'utilisateur générique du système, tandis que Customer, Employee et Admin correspondent à ses spécialisations. Cette généralisation permet de représenter la hiérarchie des acteurs tout en distinguant les fonctionnalités accessibles selon leur rôle.

Le Customer intervient principalement dans la consultation des produits, la gestion de son panier et les opérations relatives à ses propres commandes. L'Employee intervient dans les opérations de gestion qui lui sont accessibles selon les permissions qui lui sont attribuées. L'Admin dispose de droits étendus sur les fonctionnalités de gestion.

Les permissions d'accès ne sont pas représentées comme des acteurs distincts. Elles

constituent des mécanismes de contrôle d'accès permettant de déterminer les fonctionnalités accessibles à un employé.

Description des acteurs

- **User** : représente l'acteur générique dont héritent les différents types d'utilisateurs de l'application. Il permet de représenter la généralisation commune aux acteurs Customer, Employee et Admin.
- **Customer** : il peut consulter les produits, gérer son panier, confirmer une commande, consulter ses commandes et annuler une commande selon les règles métier de l'application.
- **Employee** : il accède aux fonctionnalités de gestion selon les permissions qui lui sont attribuées. Il peut notamment intervenir dans la gestion des produits, des catégories, des marques, du stock, des fournisseurs et des commandes lorsque les autorisations correspondantes lui sont accordées.
- **Admin** : il dispose de droits étendus sur les fonctionnalités de gestion. Il peut notamment gérer les produits, les catégories, les marques, le stock, les fournisseurs et les commandes.

Principaux cas d'utilisation

Les principaux cas d'utilisation du Sprint 2 sont regroupés selon les domaines fonctionnels suivants :

- **Gestion du catalogue** : consulter les produits, gérer les produits, gérer les catégories et gérer les marques. La consultation du catalogue comprend également la recherche, le filtrage, le tri et la pagination.
- **Gestion du stock** : consulter le stock, réceptionner une livraison et consulter les mouvements de stock.
- **Gestion des fournisseurs** : consulter, rechercher, ajouter, modifier et supprimer un fournisseur.
- **Gestion du panier** : consulter le panier, ajouter des produits, modifier les quantités, supprimer des articles et vider le panier.
- **Gestion des commandes** : confirmer une commande, consulter ses commandes, annuler une commande, consulter les commandes et modifier leur statut.
- **Paielement** : effectuer un paiement associé à une facture.

Diagramme de Cas d'Utilisation du Sprint 2

La figure suivante présente le diagramme de cas d'utilisation correspondant aux principales fonctionnalités développées au cours du Sprint 2.

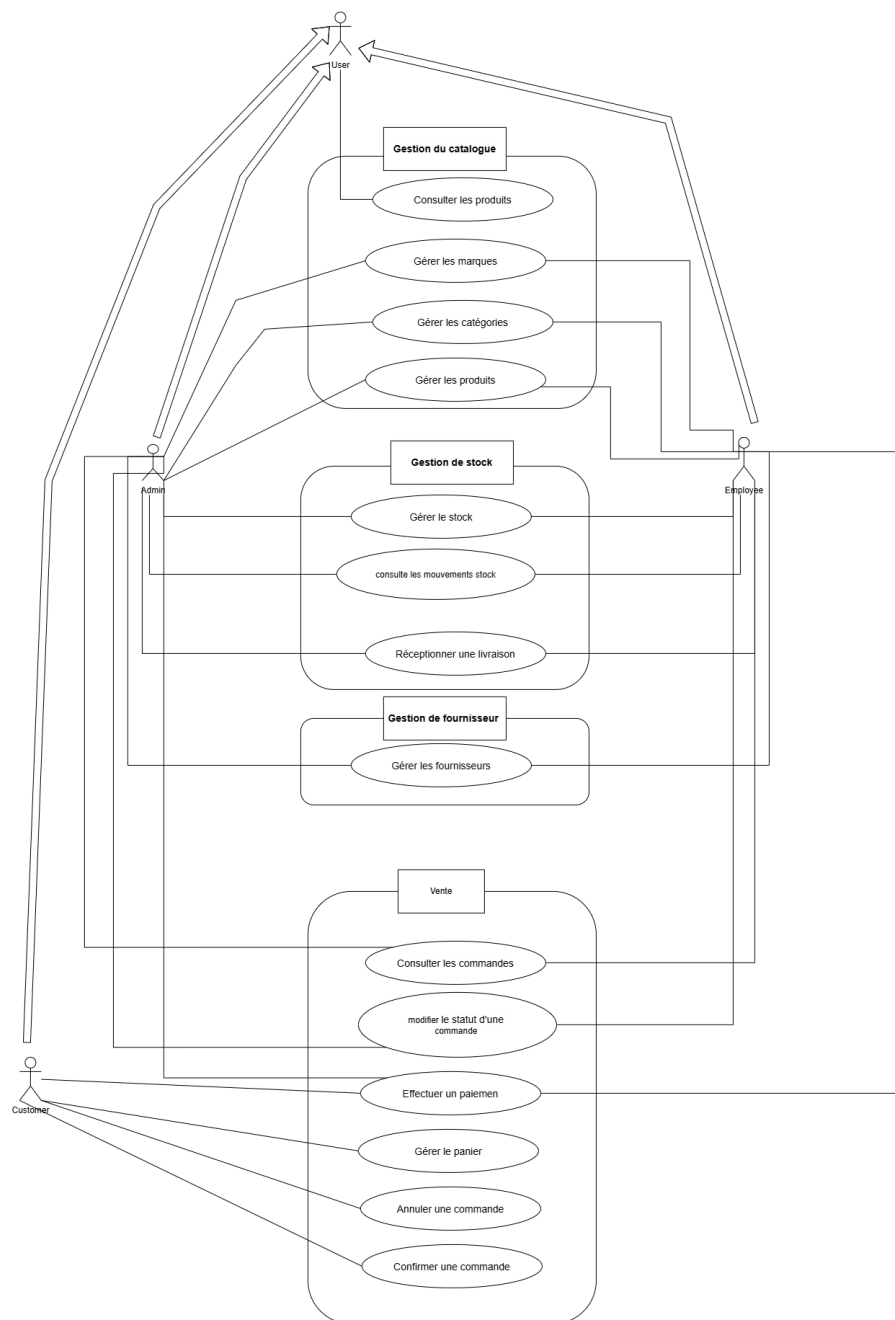


FIGURE 4.1 – Diagramme de cas d'utilisation du Sprint 2

Description des Cas d'Utilisation Principaux

UC1 : Consulter les produits Ce cas d'utilisation permet aux utilisateurs de consulter les produits disponibles dans le catalogue. La consultation peut être réalisée avec différents paramètres permettant d'organiser les résultats affichés.

UC2 : Gérer les produits Ce cas d'utilisation permet aux utilisateurs disposant des autorisations nécessaires d'ajouter, de modifier et de supprimer les produits du catalogue.

UC3 : Gérer les catégories Ce cas d'utilisation permet de consulter et d'ajouter les catégories utilisées pour organiser les produits du catalogue.

UC4 : Gérer les marques Ce cas d'utilisation permet de consulter, d'ajouter et de supprimer les marques associées aux produits.

UC5 : Gérer le stock Ce cas d'utilisation permet de consulter et de suivre les informations relatives aux quantités disponibles pour les différents produits.

UC6 : Réceptionner une livraison Ce cas d'utilisation permet d'enregistrer la réception d'une livraison provenant d'un fournisseur. Cette opération entraîne la mise à jour du stock et l'enregistrement du mouvement correspondant.

UC7 : Consulter les mouvements de stock Ce cas d'utilisation permet de consulter les mouvements enregistrés afin de suivre les opérations ayant affecté le stock.

UC8 : Gérer les fournisseurs Ce cas d'utilisation permet de consulter, rechercher, ajouter, modifier et supprimer les informations relatives aux fournisseurs.

UC9 : Gérer le panier Ce cas d'utilisation permet au client de consulter son panier, d'ajouter des produits, de modifier leurs quantités, de supprimer des articles ou de vider le panier.

UC10 : Confirmer une commande Ce cas d'utilisation permet au client de confirmer une commande à partir des produits présents dans son panier. La commande est alors enregistrée dans le système.

UC11 : Consulter ses commandes Ce cas d'utilisation permet au client de consulter les commandes qui lui sont associées et de suivre leur état.

UC12 : Annuler une commande Ce cas d'utilisation permet au client d'annuler une commande qui lui appartient, sous réserve des règles métier appliquées par l'application.

UC13 : Consulter les commandes Ce cas d'utilisation permet aux utilisateurs disposant des autorisations nécessaires de consulter les commandes enregistrées dans le système.

UC14 : Modifier le statut d'une commande Ce cas d'utilisation permet aux utilisateurs autorisés de modifier le statut d'une commande afin d'assurer son suivi au cours du processus de traitement.

UC15 : Effectuer un paiement Ce cas d'utilisation permet d'enregistrer le paiement associé à une facture selon les autorisations définies dans l'application.

4.3.2 Diagramme de Séquence

Afin d'illustrer le fonctionnement dynamique des principales fonctionnalités du Sprint 2, trois diagrammes de séquence ont été retenus. Une attention particulière est portée à la gestion du stock, qui constitue le cœur fonctionnel de ce sprint, à travers un diagramme détaillé illustrant la réception d'une livraison et la mise à jour automatique des quantités disponibles.

Réception d'une livraison et mise à jour du stock

Ce diagramme illustre le processus central de la gestion du stock : l'enregistrement d'une réception de livraison en provenance d'un fournisseur, la mise à jour du prix d'achat référence chez ce fournisseur, ainsi que la mise à jour automatique de la quantité disponible, du prix moyen pondéré et du mouvement de stock associé.

Le scénario principal est le suivant :

- L'employé saisit le produit, le fournisseur, la quantité, le prix unitaire et une note, puis valide la réception.
- Le contrôleur transmet la requête au service de gestion des approvisionnements.
- Le service vérifie l'existence du produit et du fournisseur.
- La référence fournisseur du produit (prix d'achat, date de dernière livraison) est créée ou mise à jour.
- Le stock du produit est récupéré, ou créé s'il n'existe pas encore.
- Le prix moyen pondéré et la nouvelle quantité disponible sont calculés puis enregistrés.
- Si la quantité disponible devient positive, le statut du produit est mis à jour à « En stock ».
- Un mouvement de stock de type entrée est enregistré avec le fournisseur, la quantité et le prix unitaire.
- Les informations mises à jour du stock sont retournées à l'employé.

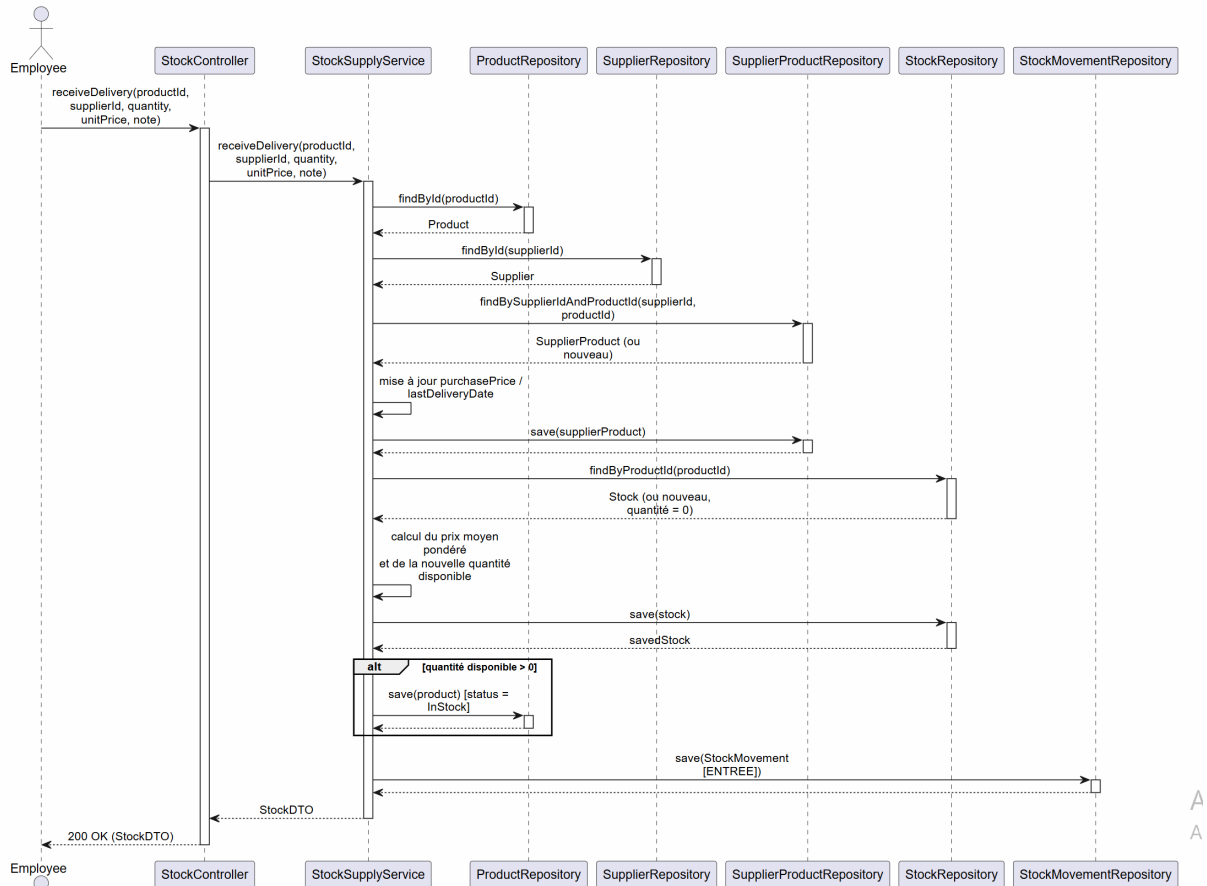


FIGURE 4.2 – Diagramme de séquence - Réception d'une livraison et mise à jour du stock

Ajustement manuel du stock

Ce diagramme illustre le processus permettant à un employé d'ajuster manuellement la quantité disponible d'un produit, par exemple à la suite d'un inventaire ou d'une correction. Chaque ajustement est automatiquement tracé sous la forme d'un mouvement de stock.

Le scénario principal est le suivant :

- L'employé sélectionne un stock et saisit une quantité d'ajustement ainsi qu'un motif.
- Le contrôleur transmet la demande au service de gestion du stock.
- Le service récupère le stock concerné et calcule la nouvelle quantité disponible.
- Si la nouvelle quantité est négative, une erreur est retournée et l'ajustement est rejeté.
- Sinon, la quantité disponible et la date de mise à jour du stock sont enregistrées.
- Un mouvement de stock est créé, de type entrée si l'ajustement est positif ou de type sortie s'il est négatif.
- La confirmation de l'ajustement est retournée à l'employé.

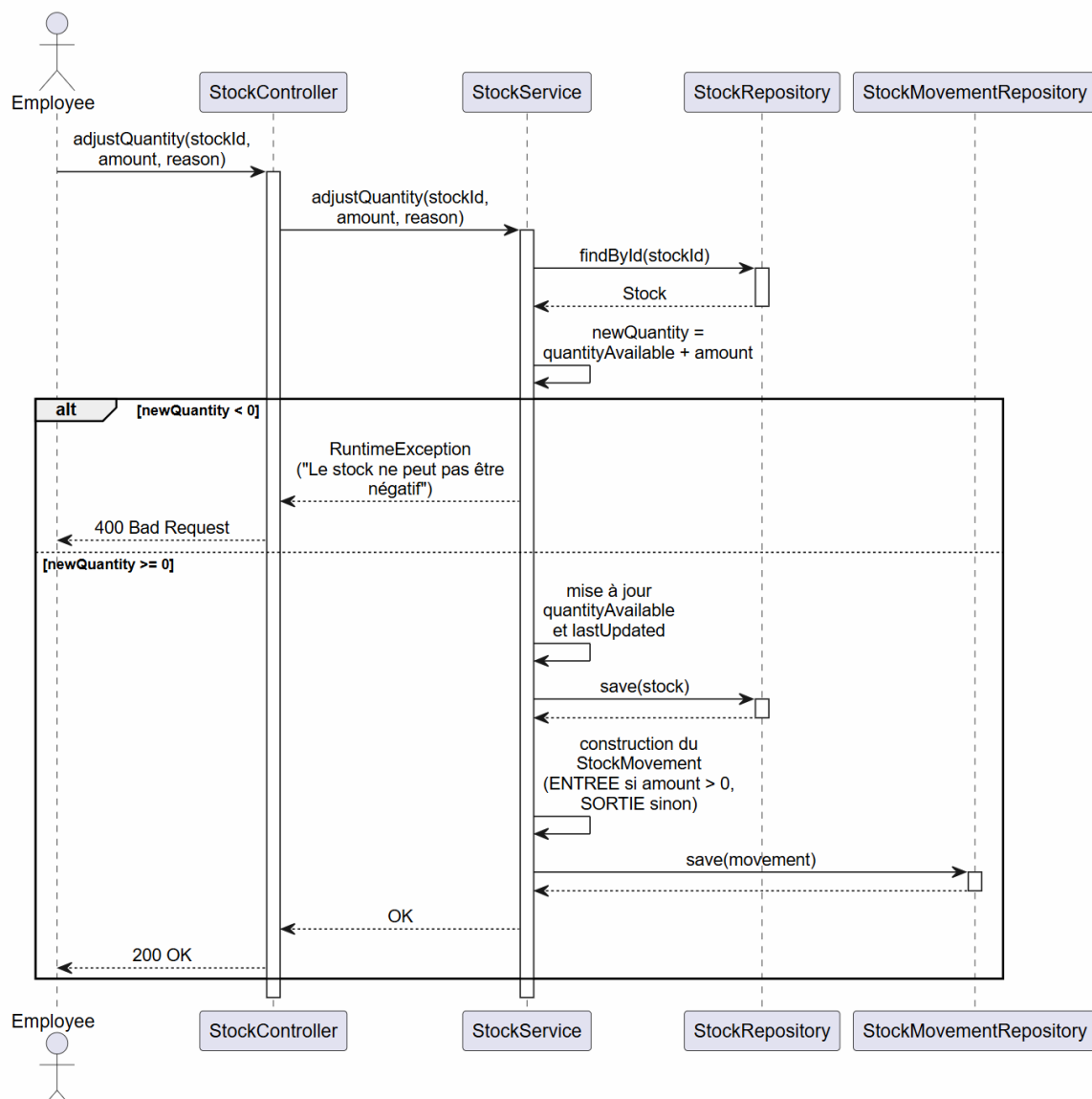


FIGURE 4.3 – Diagramme de séquence - Ajustement manuel du stock

Ajout au panier et confirmation d'une commande

Ce diagramme présente le parcours du client, depuis l'ajout d'un produit à son panier jusqu'à la confirmation de sa commande, avec la génération automatique de la facture associée.

Le scénario principal est le suivant :

- Le client ajoute un produit à son panier en précisant la quantité souhaitée.
- Le service vérifie l'existence du produit et récupère ou crée le panier du client.
- Si le produit est déjà présent dans le panier, la quantité est incrémentée ; sinon, un nouvel article est ajouté avec son prix actualisé.
- Le panier mis à jour est retourné au client.
- Lorsque le client confirme sa commande, le panier associé est récupéré.

- Si le panier est vide, la confirmation est rejetée.
- Sinon, une commande est créée à partir des articles du panier et son montant total est calculé.
- Une facture est générée et associée à la commande.
- Le panier du client est vidé.
- La confirmation de la commande est retournée au client.

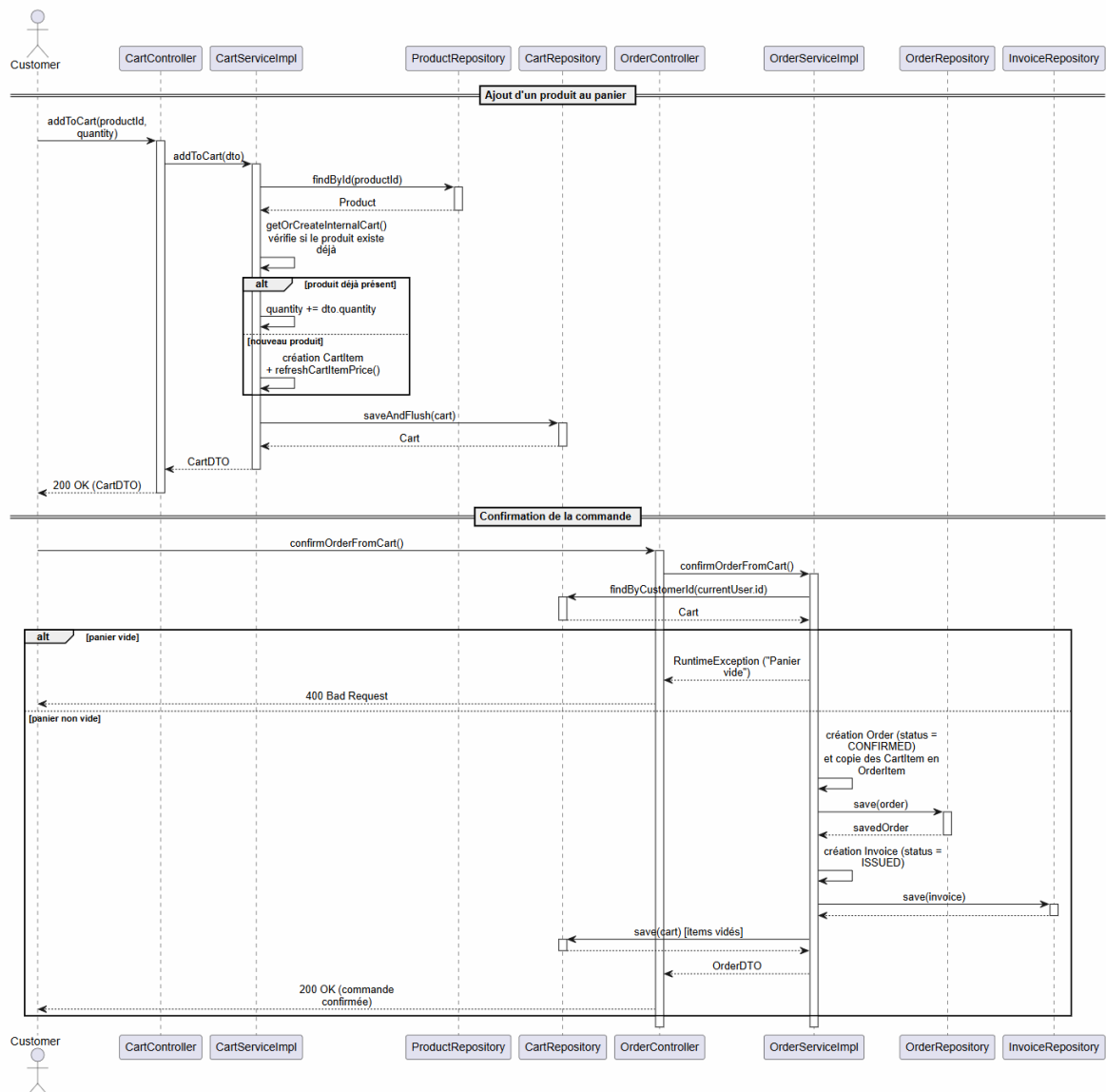


FIGURE 4.4 – Diagramme de séquence - Ajout au panier et confirmation d'une commande

4.3.3 Diagramme de classes

Le diagramme de classes ci-dessous (Figure 4.5) représente la modélisation des entités du sprint 2, dédié à la **gestion des produits**. Il est organisé en trois grandes parties complémentaires :

- **Le package Produit** : il modélise le catalogue de l'application autour de la classe abstraite **Product**, dont héritent deux types concrets, **Car** et **SparePart**, selon une stratégie d'héritage **SINGLE_TABLE** avec discriminant. Chaque produit est rattaché à une **Category** et une **Brand**.
- **Le package Stock & Fournisseur** : il gère l'approvisionnement. Chaque produit possède un **Stock** unique, dont l'évolution est tracée par des **StockMovement** (entrées/sorties). Les fournisseurs (**Supplier**) sont liés aux produits via **SupplierProduct**, qui centralise les conditions d'achat (prix, quantité minimale, fournisseur préféré). Les commandes fournisseurs (**PurchaseOrder**, **PurchaseOrderItem**) donnent lieu à une **PurchaseInvoice**, elle-même réglée via un ou plusieurs **SupplierPayment** découpés en **SupplierPaymentLine**.
- **Le package Commande, Panier & Paiement** : il couvre le parcours client, depuis le **Cart** (panier) et ses **CartItem**, jusqu'à la validation d'une **Order** composée de **OrderItem**. Chaque commande génère une **Invoice** et est réglée par un **Payment**, lui-même décomposé en plusieurs **PaymentLine** pour permettre les paiements mixtes (ex. espèces + chèque).

La classe **PaymentMethod** est mutualisée entre les deux flux financiers (client et fournisseur), garantissant une gestion homogène des moyens de paiement (espèces, chèque, carte, virement, crédit) dans l'ensemble de l'application.

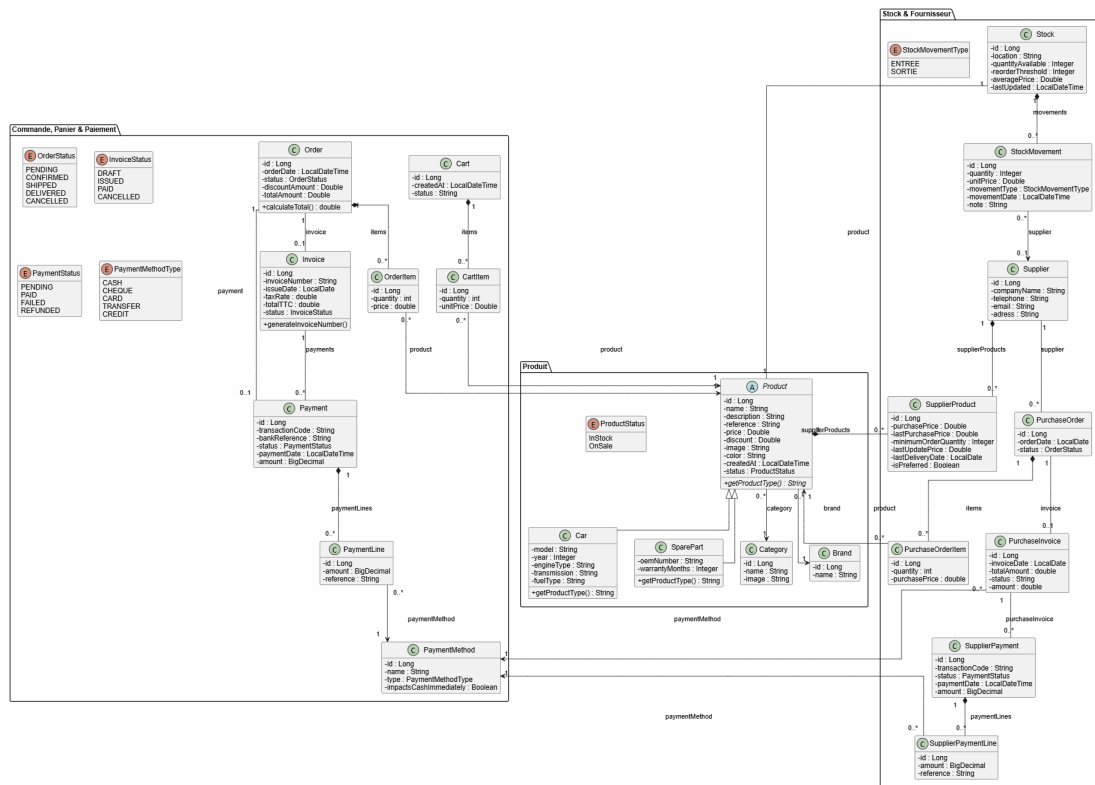


FIGURE 4.5 – Diagramme de classes du sprint 2 - Gestion des produits

4.4 Réalisation

4.4.1 Gestion des produits

Consultation et liste des produits

Recherche, filtrage et tri des produits

Ajout d'un produit

Modification d'un produit

Gestion des catégories

Gestion des marques

4.4.2 Gestion du stock

Consultation du stock

Réception d'une livraison

Ajustement des quantités

Gestion des mouvements de stock

Détection des produits en stock faible

4.4.3 Gestion des fournisseurs

Liste et recherche des fournisseurs

Ajout et modification d'un fournisseur

4.4.4 Gestion du panier et des commandes

Ajout des produits au panier

Gestion des quantités et suppression des articles

Gestion des demandes de maintenance dans le panier

Création et suivi des commandes

4.4.5 Gestion des factures et des paiements

Génération et consultation des factures

Gestion des méthodes de paiement

Traitement du paiement

Mise à jour du stock après paiement

Gestion de la caisse

4.5 Bilan du Sprint

Chapitre 5

Réalisation du Sprint 3 : Gestion des services de maintenance et des interventions

5.1 Introduction

Ce troisième sprint est consacré à la mise en place du module de gestion des services de maintenance et des interventions automobiles. Il constitue une extension de la plateforme e-commerce en intégrant les prestations de maintenance dans le système ERP.

Ce sprint permet au client de consulter les différents services de maintenance proposés, de sélectionner une prestation et de l'associer à l'un de ses véhicules. Il peut ainsi créer une demande de maintenance directement depuis l'application en renseignant le véhicule concerné ainsi que le créneau souhaité.

Du côté de l'ERP, les administrateurs peuvent gérer les services de maintenance, consulter les demandes des clients, affecter les employés aux interventions et suivre leur état d'avancement. Le système prend également en compte les horaires de travail et les interventions déjà planifiées afin de déterminer les employés disponibles.

Le module permet également de gérer les véhicules des clients. Chaque véhicule contient différentes informations techniques telles que la marque, le modèle, l'année, l'immatriculation, le numéro VIN, le kilométrage et le type de carburant. Ces informations sont utilisées lors de la création d'une demande de maintenance.

Enfin, ce sprint assure l'intégration entre le parcours e-commerce et les fonctionnalités ERP. Lors de la création d'une demande, une facture de maintenance peut être générée et le service peut être ajouté au panier du client. La demande peut ensuite être confirmée automatiquement lorsqu'un employé est disponible ou rester en attente lorsqu'aucune disponibilité n'est trouvée.

5.2 Analyse du Sprint

5.2.1 Sprint Backlog

Le Sprint Backlog regroupe les principales fonctionnalités à réaliser au cours de ce sprint. Il permet de définir les besoins fonctionnels liés à la gestion des services de maintenance, des véhicules clients, des demandes d'intervention, de leur planification ainsi que de leur intégration avec le panier et la facturation.

ID	User Story	Critère d'acceptation	Priorité
US1	En tant que client, je souhaite consulter les services de maintenance afin de choisir une prestation adaptée à mon véhicule.	Les services sont affichés avec leur nom, leur description, leur prix, leur durée estimée et leur image.	Haute
US2	En tant qu'administrateur, je souhaite gérer les services de maintenance afin de maintenir le catalogue des prestations à jour.	Un service peut être ajouté, modifié ou supprimé et peut être associé à un département.	Haute
US3	En tant que client, je souhaite enregistrer mon véhicule afin de pouvoir l'utiliser lors d'une demande de maintenance.	Les informations du véhicule sont enregistrées et associées au client concerné.	Haute
US4	En tant que client, je souhaite consulter mes véhicules afin de sélectionner celui qui sera concerné par l'intervention.	Les véhicules associés au client connecté sont affichés avec leurs principales informations.	Haute
US5	En tant que client, je souhaite demander un service de maintenance pour mon véhicule afin de réserver une intervention.	Une demande est créée avec le service, le véhicule, la date de demande et le créneau souhaité.	Haute

ID	User Story	Critère d'acceptation	Priorité
US6	En tant qu'administrateur, je souhaite créer une demande de maintenance pour un client afin d'enregistrer une intervention dans l'ERP.	Une demande est créée pour le véhicule et le service sélectionnés et son état est enregistré.	Haute
US7	En tant qu'employé, je souhaite consulter les demandes de maintenance afin de suivre les interventions qui me sont attribuées.	Les demandes sont affichées avec les informations du client, du véhicule, du service et leur statut.	Haute
US8	En tant que système, je souhaite rechercher automatiquement un employé disponible afin d'affecter une intervention au créneau demandé.	Un employé actif et disponible appartenant au département concerné est affecté lorsque les conditions sont respectées.	Haute
US9	En tant que système, je souhaite rechercher un créneau proche lorsqu'aucun employé n'est disponible afin de proposer une alternative.	Un créneau disponible est recherché dans la période définie et la demande est confirmée lorsqu'un employé peut être affecté.	Haute
US10	En tant qu'administrateur, je souhaite affecter manuellement un employé à une demande afin de gérer les interventions.	Un employé est affecté à la demande et un créneau de travail correspondant est enregistré.	Haute
US11	En tant qu'employé ou administrateur, je souhaite modifier le statut d'une demande afin de suivre son avancement.	Le statut de la demande évolue selon les états prévus par le système.	Haute

ID	User Story	Critère d'acceptation	Priorité
US12	En tant que client, je souhaite consulter mes demandes de maintenance afin de suivre l'état de mes interventions.	Les demandes associées aux véhicules du client sont affichées avec leur statut.	Moyenne
US13	En tant que système, je souhaite générer une facture lors de la création d'une demande afin d'assurer le suivi financier de la prestation.	Une facture de maintenance est créée et son numéro est associé à la demande.	Haute
US14	En tant que système, je souhaite ajouter automatiquement le service de maintenance au panier du client afin d'intégrer la prestation au processus commercial.	Le service est ajouté au panier avec une quantité de 1 et le prix correspondant au service demandé.	Haute

TABLE 5.1 – Sprint Backlog du Sprint 3

5.3 Conception

5.3.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation du Sprint 3 présente les interactions entre les différents acteurs du système et les fonctionnalités liées aux services de maintenance et aux interventions.

Dans ce sprint, trois acteurs principaux sont considérés : **Customer**, **Employee** et **Admin**. Le Customer intervient principalement dans la consultation des services, la gestion de ses véhicules, la création de demandes de maintenance et le suivi de ses interventions.

L'Employee intervient dans le traitement des interventions qui lui sont affectées. Il peut consulter les demandes qui lui sont attribuées et mettre à jour leur état selon les autorisations qui lui sont accordées.

L'Admin dispose de droits étendus sur le module de maintenance. Il peut gérer les services, consulter les véhicules et les demandes, créer une demande pour un client et affecter manuellement un employé à une intervention.

Description des acteurs

- **Customer** : il peut consulter les services de maintenance, gérer ses véhicules, créer une demande de maintenance pour l'un de ses véhicules et consulter ses demandes afin de suivre leur état.
- **Employee** : il intervient dans le traitement des demandes de maintenance qui lui sont affectées. Il peut consulter les interventions et mettre à jour leur statut au cours de leur réalisation.
- **Admin** : il dispose de droits étendus sur le module de maintenance. Il peut gérer les services, consulter les véhicules et les demandes, créer des demandes pour les clients, affecter manuellement des employés et superviser les interventions.

Principaux cas d'utilisation

Les principaux cas d'utilisation du Sprint 3 sont regroupés selon les domaines fonctionnels suivants :

- **Gestion des services de maintenance** : consulter, rechercher, filtrer, ajouter, modifier et supprimer les services de maintenance.
- **Gestion des véhicules clients** : ajouter, modifier, consulter et supprimer les véhicules associés aux clients.
- **Gestion des demandes** : créer et consulter les demandes de maintenance associées aux véhicules et aux services.
- **Affectation des interventions** : rechercher automatiquement un employé disponible ou permettre à l'administrateur d'effectuer une affectation manuelle.
- **Planification** : rechercher un créneau disponible en fonction du service, du département, des horaires de travail et des interventions déjà planifiées.
- **Suivi des interventions** : modifier et consulter le statut d'une demande pendant son cycle de traitement.
- **Intégration commerciale** : ajouter le service de maintenance au panier et générer la facture associée à la demande.

Diagramme de Cas d'Utilisation du Sprint 3

La figure suivante présente le diagramme de cas d'utilisation correspondant aux principales fonctionnalités développées au cours du Sprint 3.

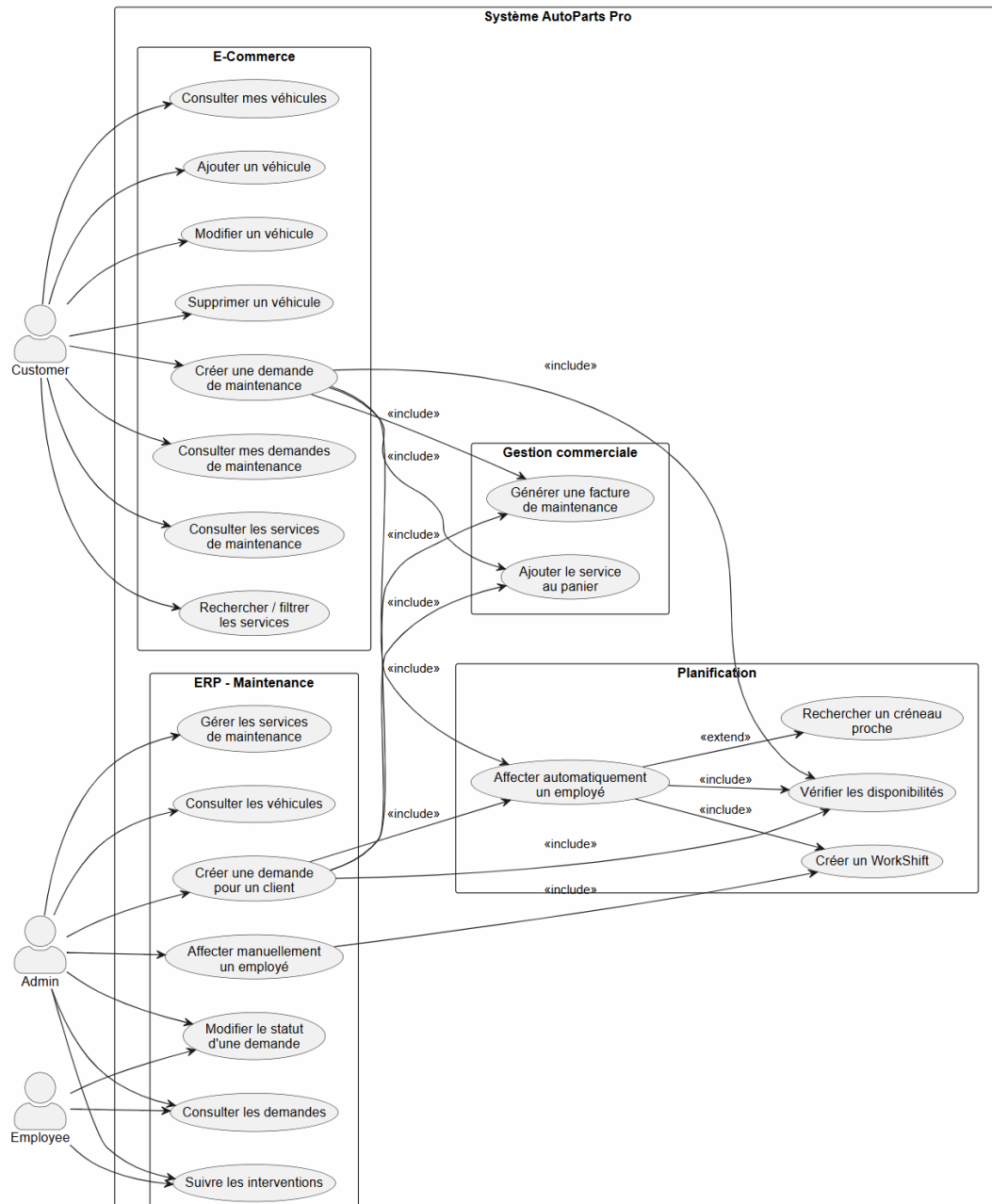


FIGURE 5.1 – Diagramme de cas d'utilisation du Sprint 3

Description des Cas d'Utilisation Principaux

UC1 : Consulter les services de maintenance Ce cas d'utilisation permet au client de consulter les prestations de maintenance disponibles. Les informations associées à chaque prestation comprennent notamment son nom, sa description, son prix, sa durée estimée et son image.

UC2 : Gérer les services de maintenance Ce cas d'utilisation permet à l'administrateur de gérer le catalogue des services de maintenance. Il comprend l'ajout, la modification

et la suppression des prestations ainsi que leur association à un département.

UC3 : Gérer les véhicules clients Ce cas d'utilisation permet au client d'enregistrer et de gérer ses véhicules. Chaque véhicule contient les principales informations nécessaires à son identification et à son suivi dans le cadre des interventions de maintenance.

UC4 : Consulter les véhicules Ce cas d'utilisation permet au client de consulter ses propres véhicules. L'administrateur peut également consulter l'ensemble des véhicules enregistrés dans le système afin d'assurer leur gestion dans l'ERP.

UC5 : Créer une demande de maintenance Ce cas d'utilisation permet au client de sélectionner un véhicule, un service et un créneau souhaité afin de créer une demande de maintenance.

Le système enregistre la date de création de la demande et recherche automatiquement un employé disponible pour le créneau demandé.

UC6 : Créer une demande pour un client Ce cas d'utilisation permet à l'administrateur de créer une demande de maintenance pour un client donné. Le véhicule et le service sont sélectionnés puis le système recherche les disponibilités nécessaires à la planification de l'intervention.

UC7 : Consulter les demandes de maintenance Ce cas d'utilisation permet aux utilisateurs autorisés de consulter les demandes enregistrées dans le système.

Les informations affichées comprennent notamment le client, son numéro de téléphone, le véhicule, le service, l'employé affecté, le numéro de facture et le statut de la demande.

UC8 : Affecter automatiquement un employé Ce cas d'utilisation permet au système de rechercher automatiquement un employé pouvant réaliser l'intervention.

La recherche prend en compte le département du service, les horaires de travail de l'employé, son état actif et sa disponibilité ainsi que les autres créneaux déjà planifiés.

UC9 : Rechercher un créneau proche Lorsque aucun employé n'est disponible pour le créneau initialement demandé, le système recherche automatiquement un créneau alternatif proche.

La recherche est effectuée sur une période pouvant aller jusqu'à trente jours et permet de trouver un créneau correspondant à la durée de l'intervention.

UC10 : Affecter manuellement un employé Ce cas d'utilisation permet à l'administrateur de sélectionner manuellement un employé pour une demande de maintenance.

Un `WorkShift` correspondant au rendez-vous est alors créé avec le département, les horaires et l'employé sélectionné.

UC11 : Modifier le statut d'une demande Ce cas d'utilisation permet de modifier le statut d'une demande de maintenance.

Les différents états prévus sont `REQUESTED`, `CONFIRMED`, `IN_PROGRESS`, `COMPLETED` et `CANCELLED`.

UC12 : Consulter ses demandes Ce cas d'utilisation permet au client de consulter les demandes associées à ses véhicules afin de suivre le déroulement de ses interventions.

UC13 : Générer une facture de maintenance Ce cas d'utilisation permet de créer automatiquement une facture lors de l'enregistrement d'une demande de maintenance.

Le numéro de facture généré est ensuite associé à la demande.

UC14 : Ajouter le service au panier Ce cas d'utilisation permet d'intégrer la prestation de maintenance au panier du client. Un `CartItem` est créé avec la demande de maintenance, une quantité de un et le prix du service.

5.3.2 Diagramme de Séquence

Afin d'illustrer le fonctionnement dynamique des principales fonctionnalités du Sprint 3, trois diagrammes de séquence ont été retenus. Ils représentent respectivement la création d'une demande de maintenance, l'affectation automatique et la planification d'une intervention, puis le traitement et le suivi de celle-ci.

Création d'une demande de maintenance

Ce diagramme présente le parcours du client depuis la sélection d'un service et d'un véhicule jusqu'à l'enregistrement de la demande.

Le scénario principal est le suivant :

- Le client sélectionne un service de maintenance.
- Le client sélectionne l'un de ses véhicules.
- Le client renseigne le créneau souhaité pour l'intervention.
- La requête est transmise au contrôleur de maintenance.
- Le service récupère le client connecté à partir de son adresse e-mail.
- Le véhicule sélectionné est récupéré à partir de son identifiant.
- Le service de maintenance est récupéré à partir de son identifiant.
- Le système vérifie que le service est associé à un département.
- Une nouvelle instance de `MaintenanceRequest` est créée.

- La demande est associée au véhicule et au service sélectionnés.
- La date de création ainsi que les horaires demandés sont enregistrés.
- Le système recherche automatiquement un employé disponible.
- La demande est enregistrée avec le statut approprié.
- Une facture de maintenance est générée.
- Le numéro de facture est associé à la demande.
- Le service de maintenance est ajouté au panier du client.
- La demande enregistrée est retournée au client.

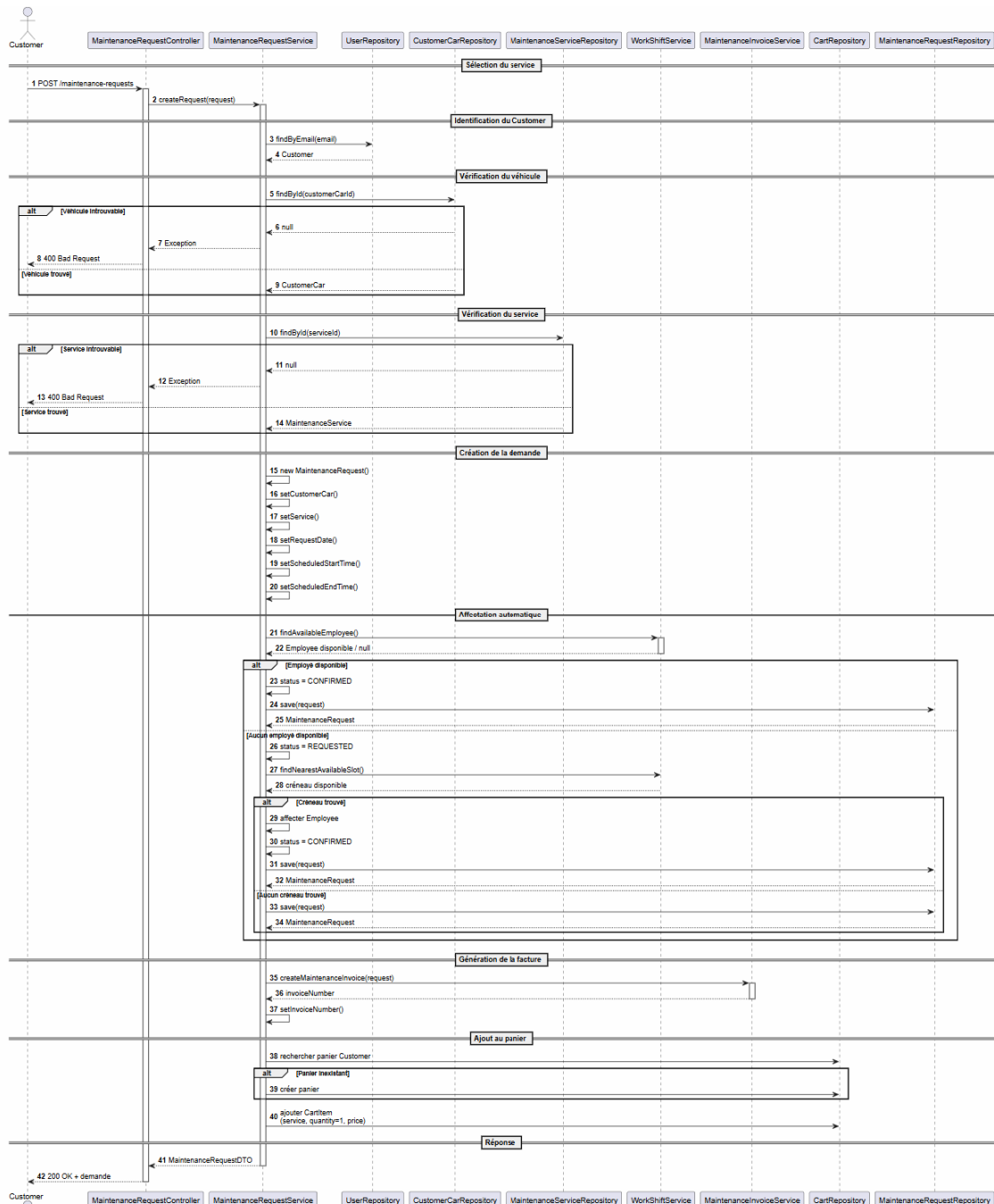


FIGURE 5.2 – Diagramme de séquence - Création d'une demande de maintenance

Affectation automatique et planification d'une intervention

Ce diagramme présente le mécanisme permettant de rechercher automatiquement un employé disponible pour réaliser l'intervention.

Le scénario principal est le suivant :

- Le système récupère le département associé au service demandé.
- Le service de planification recherche les employés correspondant au département.
- Les horaires de travail sont vérifiés afin de déterminer si le créneau demandé est compatible avec les horaires de l'employé.
- Les employés inactifs ou indisponibles sont exclus.
- Les **WorkShift** existants sont analysés afin d'éviter les conflits de planning.
- Si un employé est disponible, il est automatiquement affecté à la demande.
- Un **WorkShift** correspondant à l'intervention est enregistré.
- La demande passe au statut **CONFIRMED**.
- Si aucun employé n'est disponible, la demande reste au statut **REQUESTED**.
- Le système recherche alors un créneau proche correspondant à la durée de l'intervention.
- Lorsqu'un créneau disponible est trouvé, un employé est affecté et la demande est confirmée.

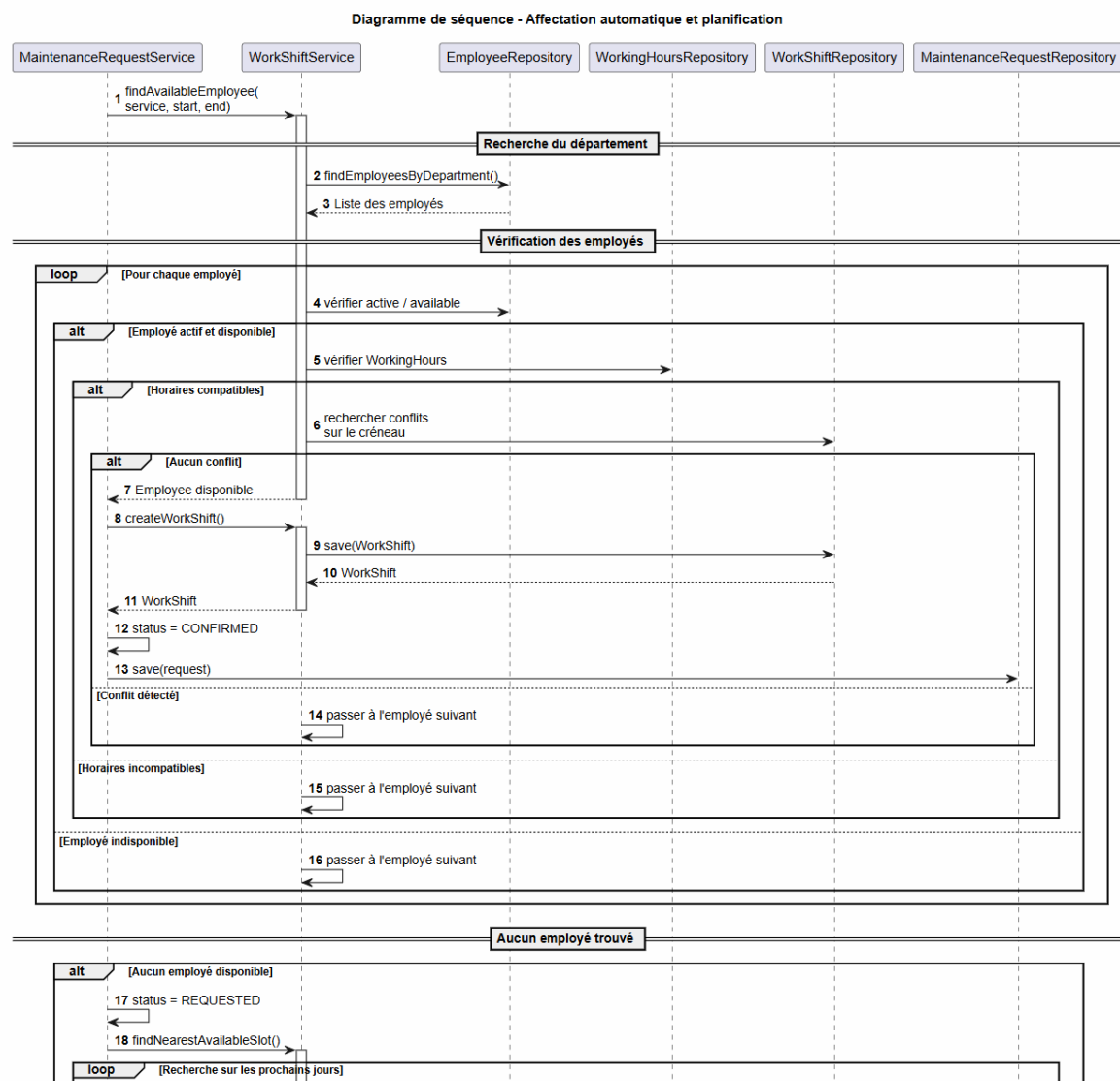


FIGURE 5.3 – Diagramme de séquence - Affectation automatique et planification d’une intervention

Traitement et suivi d’une intervention

Ce diagramme illustre le traitement d’une demande de maintenance par l’employé affecté ou par l’administrateur.

Le scénario principal est le suivant :

- L’employé consulte les interventions qui lui sont affectées.
- L’employé démarre l’intervention.
- Le statut de la demande passe à `IN_PROGRESS`.
- L’employé réalise les opérations de maintenance prévues.
- Lorsque l’intervention est terminée, le statut passe à `COMPLETED`.
- L’administrateur peut également modifier le statut d’une demande lorsque cela est

nécessaire.

- Le client peut consulter le statut actualisé de sa demande depuis son espace.

Le système applique un contrôle d'accès lors de la modification du statut. Un employé ne peut modifier que les demandes qui lui sont affectées, tandis que l'administrateur dispose de droits plus larges sur les demandes.

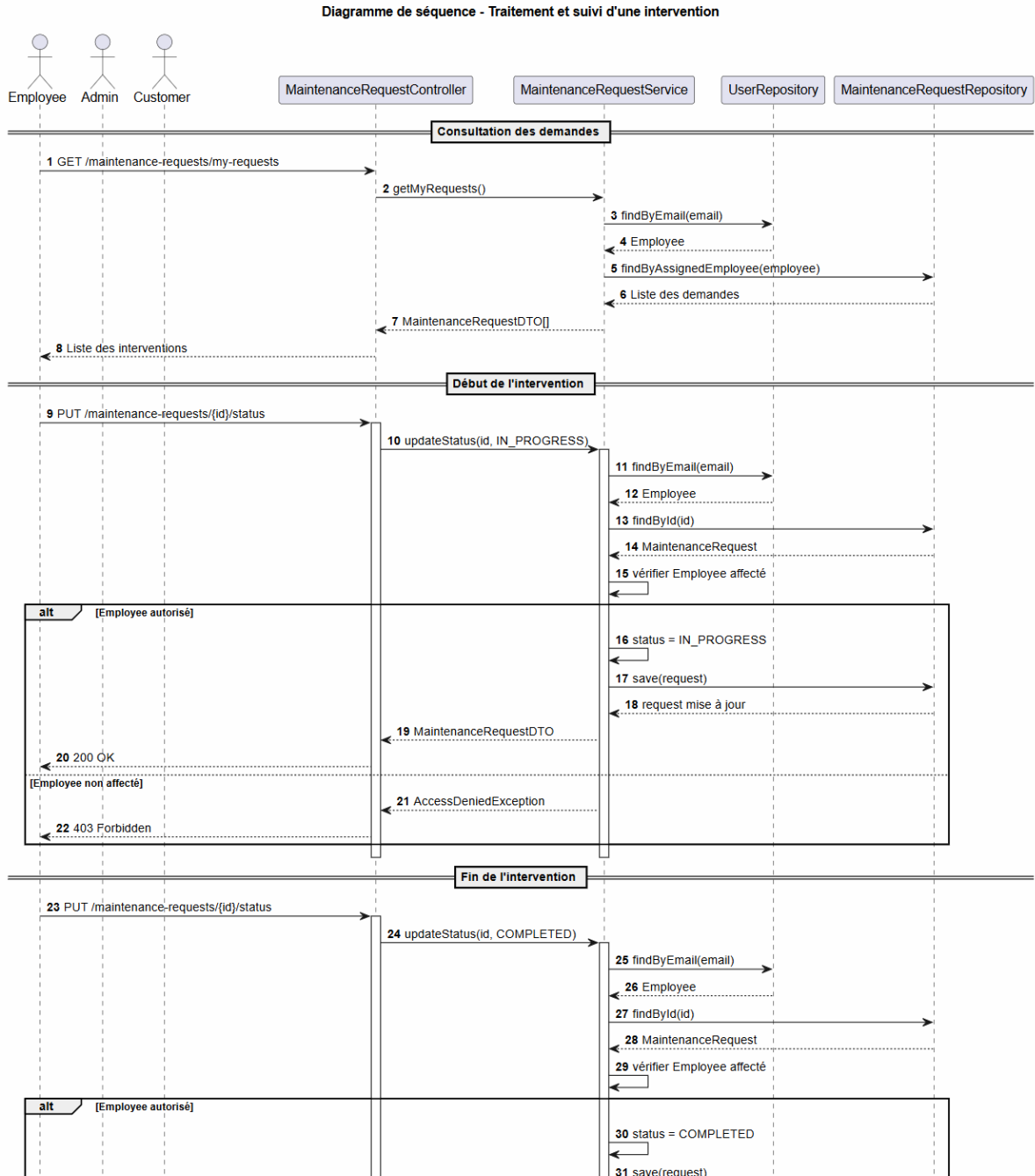


FIGURE 5.4 – Diagramme de séquence - Traitement et suivi d'une intervention

5.3.3 Diagramme de Classes

Le diagramme de classes ci-dessous représente les principales entités du Sprint 3, dédié à la gestion des services de maintenance et des interventions.

Il est organisé autour de plusieurs éléments complémentaires :

- **Le package Service de maintenance** : il est représenté par la classe `MaintenanceService`. Cette classe contient le nom, la description, le prix, la durée estimée et l'image du service. Elle est également associée à un `Department`.
- **Le package Véhicule client** : il est représenté par la classe `CustomerCar`. Cette classe permet d'enregistrer les informations du véhicule, notamment sa marque, son modèle, son année, son immatriculation, son VIN, son kilométrage et son carburant. Elle est associée à un `Customer`.
- **Le package Demande de maintenance** : il est représenté par la classe `MaintenanceRequest`. Cette classe constitue le lien entre le véhicule du client et le service demandé. Elle contient également la date de demande, le statut, l'employé affecté, les horaires planifiés et le numéro de facture.
- **Le package Planification** : les informations de planification sont représentées par les horaires de début et de fin de l'intervention. La classe `ProposedSlot` permet également de représenter un créneau proposé lorsqu'une proposition de rendez-vous est nécessaire.
- **Le package Ressources humaines et planning** : les employés et leurs `WorkShift` sont utilisés pour déterminer les disponibilités et éviter les conflits de planning lors de l'affectation d'une intervention.
- **Le suivi de la demande** : l'énumération `MaintenanceStatus` représente les différents états d'une demande : `REQUESTED`, `CONFIRMED`, `IN_PROGRESS`, `COMPLETED` et `CANCELLED`.

La modélisation permet ainsi de relier le parcours du client à la gestion opérationnelle de l'ERP. Une demande de maintenance relie un client, son véhicule, un service de maintenance et, lorsqu'une disponibilité est trouvée, un employé chargé de l'intervention.

Diagramme de classes - Sprint 3 : Gestion des services de maintenance

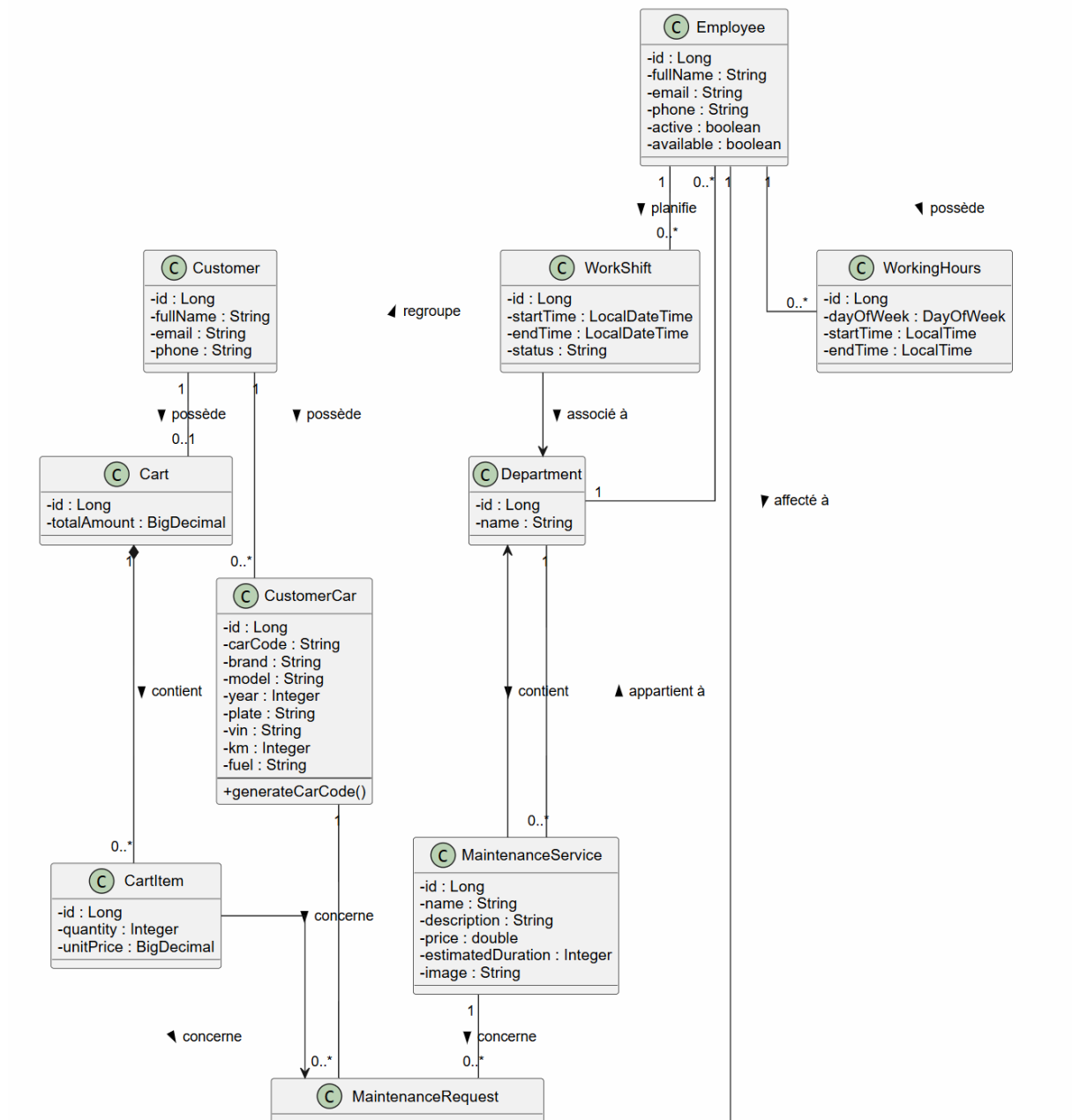


FIGURE 5.5 – Diagramme de classes du Sprint 3 - Gestion des services de maintenance

5.4 Réalisation

5.4.1 Gestion des services de maintenance

Consultation des services de maintenance

La consultation des services de maintenance permet aux clients d'accéder au catalogue des prestations disponibles.

Chaque service contient plusieurs informations permettant au client de choisir une

prestation adaptée à ses besoins, notamment le nom, la description, le prix, la durée estimée et l'image.

La récupération des services prend également en charge la recherche, le filtrage par prix et la pagination afin de faciliter la navigation dans le catalogue.

Ajout et modification d'un service

L'administrateur peut ajouter un nouveau service de maintenance en renseignant son nom, sa description, son prix et sa durée estimée.

Un service peut également être associé à un département. Cette association est importante pour le processus d'affectation automatique, car le système utilise le département du service pour rechercher les employés capables de réaliser l'intervention.

Lors d'une modification, les informations du service sont mises à jour dans la base de données.

Gestion des images des services

Chaque service peut être associé à une image afin d'améliorer sa présentation dans l'interface e-commerce.

Lorsqu'une nouvelle image est chargée, le système peut supprimer l'ancienne image locale avant d'enregistrer la nouvelle.

Cette fonctionnalité permet de maintenir les ressources associées au catalogue de maintenance à jour.

Suppression d'un service

L'administrateur peut supprimer un service de maintenance.

Lorsqu'une image locale est associée au service, celle-ci est également supprimée afin d'éviter la conservation de fichiers qui ne sont plus utilisés.

5.4.2 Gestion des véhicules clients

Ajout d'un véhicule

Le client peut enregistrer un véhicule depuis son espace personnel.

Les informations enregistrées comprennent notamment la marque, le modèle, l'année, l'immatriculation, le numéro VIN, le kilométrage et le type de carburant.

Le véhicule est associé au client concerné et possède un code véhicule permettant de l'identifier dans le système.

Consultation des véhicules

Le client peut consulter les véhicules qui lui appartiennent afin de sélectionner celui qui sera utilisé lors d'une demande de maintenance.

L'administrateur peut également consulter l'ensemble des véhicules enregistrés dans l'ERP afin d'accéder aux informations nécessaires au traitement des demandes.

Modification d'un véhicule

Le propriétaire peut modifier les informations de son véhicule afin de maintenir les données techniques à jour.

Les informations peuvent notamment concerner le kilométrage, l'immatriculation, le modèle ou les autres caractéristiques enregistrées.

Suppression d'un véhicule

Le client peut supprimer un véhicule qui lui appartient.

Avant la suppression, le système vérifie que le véhicule est bien associé au client connecté. Cette vérification permet d'empêcher un utilisateur de supprimer le véhicule d'un autre client.

5.4.3 Gestion des demandes de maintenance

Création d'une demande par le Customer

Le client peut créer une demande de maintenance directement depuis l'application.

Il sélectionne un service de maintenance, son véhicule et un créneau souhaité pour l'intervention.

Le système récupère automatiquement le client connecté à partir de son adresse e-mail, puis vérifie l'existence du véhicule et du service.

Une nouvelle `MaintenanceRequest` est créée avec la date de demande, le véhicule, le service et les horaires souhaités.

Le système recherche ensuite automatiquement un employé disponible.

Création d'une demande par l'Admin

L'administrateur peut également créer une demande de maintenance pour un client.

Dans ce cas, le client concerné est identifié à partir de son identifiant, puis son véhicule et le service de maintenance sont associés à la nouvelle demande.

Le même mécanisme de recherche de disponibilité peut ensuite être utilisé pour déterminer l'employé chargé de l'intervention.

Consultation des demandes

Les demandes de maintenance regroupent toutes les informations nécessaires au suivi d'une intervention.

Les informations retournées comprennent notamment la date de demande, le statut, le nom et le téléphone du client, le véhicule concerné, le service sélectionné, l'employé affecté et le numéro de facture.

Les utilisateurs autorisés peuvent ainsi disposer d'une vision complète de chaque intervention.

5.4.4 Planification et affectation des interventions

Vérification des disponibilités

La disponibilité d'un employé est déterminée à partir de plusieurs critères.

Le système vérifie tout d'abord que l'employé appartient au département correspondant au service demandé.

Il vérifie ensuite que le créneau demandé respecte les horaires de travail de l'employé.

Les employés inactifs ou indisponibles sont exclus de la recherche.

Enfin, les **WorkShift** déjà planifiés sont analysés afin de vérifier qu'aucun conflit n'existe avec le nouveau créneau.

Affectation automatique d'un employé

Lorsqu'un employé répond à toutes les conditions de disponibilité, il est automatiquement affecté à la demande.

La demande passe alors au statut **CONFIRMED**.

Un **WorkShift** est créé pour représenter l'intervention planifiée. Il contient la date, les horaires, l'employé, le département et l'état du créneau.

Cette automatisation permet de réduire les interventions manuelles nécessaires à la planification des rendez-vous.

Recherche d'un créneau proche

Lorsque aucun employé n'est disponible pour le créneau demandé, la demande reste initialement au statut **REQUESTED**.

Le système recherche alors automatiquement un créneau alternatif proche en utilisant la durée de l'intervention.

La recherche est réalisée progressivement à partir du lendemain du créneau demandé et peut s'étendre sur une période allant jusqu'à trente jours.

Lorsqu'un créneau disponible est trouvé et qu'un employé peut y être affecté, le nouveau créneau est enregistré et la demande passe au statut **CONFIRMED**.

Affectation manuelle par l'Admin

L'administrateur peut également effectuer une affectation manuelle lorsqu'il souhaite choisir directement l'employé responsable de l'intervention.

Le système vérifie que l'utilisateur sélectionné est bien un employé et que le créneau de la demande est défini.

Un `WorkShift` est ensuite créé pour l'employé sélectionné et la demande passe au statut `CONFIRMED`.

5.4.5 Gestion du cycle de vie des demandes

Statut d'une demande

Le cycle de vie d'une demande de maintenance est représenté par l'énumération `MaintenanceStatus`.

Les différents états sont les suivants :

- `REQUESTED` : la demande a été enregistrée mais aucune intervention n'est encore confirmée.
- `CONFIRMED` : un employé et un créneau ont été affectés à l'intervention.
- `IN_PROGRESS` : l'intervention de maintenance est en cours.
- `COMPLETED` : l'intervention est terminée.
- `CANCELLED` : la demande a été annulée.

Contrôle des droits lors de la modification du statut

La modification du statut est soumise à un contrôle d'accès.

Lorsqu'un utilisateur souhaite modifier le statut d'une demande, le système vérifie son identité.

L'administrateur dispose des droits nécessaires pour modifier le statut des demandes.

Pour un employé, le système vérifie que celui-ci est bien l'employé affecté à la demande.

Cette vérification permet d'empêcher un employé de modifier le statut d'une intervention qui ne lui est pas attribuée.

5.4.6 Intégration avec l'e-commerce et l'ERP

Ajout du service de maintenance au panier

Lorsqu'une demande est créée pour un client, le service de maintenance est automatiquement ajouté à son panier.

Si le client ne possède pas encore de panier actif, un nouveau panier est créé.

Un `CartItem` est ensuite créé avec la demande de maintenance, une quantité égale à un et le prix du service sélectionné.

Cette intégration permet de considérer la prestation de maintenance comme un élément du processus commercial de la plateforme.

Génération de la facture de maintenance

Lors de la création d'une demande, le système fait appel au service de facturation de la maintenance afin de générer une facture.

Le numéro de facture généré est ensuite enregistré dans l'attribut `invoiceNumber` de la demande.

Cette association permet de relier directement l'intervention de maintenance à son document de facturation dans l'ERP.

Gestion des employés et des horaires

La planification des interventions repose sur les informations relatives aux employés, à leurs horaires de travail et aux créneaux déjà planifiés.

Les `WorkingHours` permettent de déterminer les périodes pendant lesquelles un employé peut intervenir.

Les `WorkShift` permettent quant à eux d'enregistrer les interventions déjà planifiées et d'éviter les conflits de disponibilité.

Cette organisation permet au système de prendre en compte les contraintes réelles de l'activité de maintenance.

5.5 Bilan du Sprint

Ce troisième sprint a permis d'intégrer un module complet de gestion des services de maintenance au sein de la plateforme AutoParts Pro.

Du côté e-commerce, le client peut consulter les services disponibles, enregistrer ses véhicules et créer une demande de maintenance en sélectionnant le véhicule concerné ainsi qu'un créneau souhaité.

Du côté ERP, les administrateurs peuvent gérer les services, consulter les demandes, créer des demandes pour les clients et affecter les employés aux interventions. Le système prend également en compte les horaires de travail et les interventions déjà planifiées afin d'automatiser la recherche des disponibilités.

L'affectation automatique permet de confirmer directement une demande lorsqu'un employé est disponible. Dans le cas contraire, le système recherche un créneau alternatif afin de faciliter la planification de l'intervention.

L'intégration avec le panier et la facturation permet également d'assurer la continuité entre le parcours e-commerce et les fonctionnalités commerciales de l'ERP.

Ainsi, ce sprint permet de mettre en place le cycle complet d'une prestation de maintenance, depuis la sélection du service par le client jusqu'à la planification, l'affectation de l'employé, le suivi de l'intervention et son intégration avec les processus commerciaux et financiers de l'entreprise.

Chapitre 6

Réalisation du Sprint 4 : Gestion des Ressources Humaines

6.1 Introduction

Ce quatrième sprint a pour objectif de mettre en place les fonctionnalités liées à la gestion des ressources humaines au sein de l'application MSG.

Il permet d'assurer la gestion et le suivi des employés à travers plusieurs fonctionnalités, notamment la vérification de leur identité par reconnaissance du numéro CIN, la gestion des horaires de travail, l'affectation des shifts, ainsi que la gestion des congés et des avances sur salaire.

Ce sprint intègre également des règles métier permettant de contrôler la disponibilité des employés, leur affectation aux créneaux de travail et les conditions associées aux demandes de congé et d'avance sur salaire. La vérification des employés repose également sur l'intégration d'un service OCR permettant d'extraire automatiquement le numéro CIN à partir des documents d'identité fournis.

6.2 Analyse du Sprint

Cette section présente l'analyse du quatrième sprint consacré à la gestion des ressources humaines.

Elle décrit les fonctionnalités sélectionnées dans le Sprint Backlog ainsi que les principaux cas d'utilisation réalisés afin de répondre aux besoins de l'administrateur et des employés.

6.2.1 Sprint Backlog

Le Sprint Backlog regroupe l'ensemble des fonctionnalités sélectionnées pour le quatrième sprint.

Chaque fonctionnalité est exprimée sous forme de User Story afin de décrire le besoin de l'utilisateur et la valeur apportée par la fonctionnalité développée.

L'objectif principal de ce sprint est de proposer un module RH permettant de gérer les employés, leur disponibilité, leurs horaires, leurs shifts, leurs congés et leurs demandes d'avance sur salaire.

ID	User Story	Critère d'acceptation	Priorité
US1	En tant qu'Admin, je souhaite vérifier l'identité d'un employé à partir de son CIN afin de confirmer son identité.	Le système extrait le numéro CIN à partir de l'image fournie, recherche l'employé correspondant et valide son compte si le CIN est reconnu.	Haute
US2	En tant qu'Admin, je souhaite vérifier automatiquement le CIN d'un employé à l'aide d'un service OCR afin de faciliter le processus de vérification.	Le service OCR extrait le numéro CIN et retourne un niveau de confiance associé au résultat.	Haute
US3	En tant qu'Admin, je souhaite consulter et gérer les horaires de travail des employés afin d'organiser leur disponibilité.	Les horaires sont enregistrés avec le jour, l'heure de début, l'heure de fin et le département concerné.	Haute
US4	En tant qu'Admin, je souhaite modifier ou supprimer les horaires d'un employé afin de maintenir son planning à jour.	Les horaires existants peuvent être modifiés ou supprimés puis enregistrés dans la base de données.	Moyenne
US5	En tant qu'Admin, je souhaite créer un shift pour un employé afin de planifier son intervention sur un créneau donné.	Le shift est enregistré avec une date, une heure de début, une heure de fin, un employé, un département et le statut PLANNED.	Haute

US6	En tant que système, je souhaite affecter automatiquement un employé disponible à une demande afin d'optimiser la planification des interventions.	Le système sélectionne un employé actif et disponible dont l'horaire couvre le créneau demandé et qui ne possède aucun shift en conflit.	Haute
US7	En tant qu'Employee, je souhaite consulter mes shifts afin de connaître mes créneaux de travail planifiés.	L'employé peut consulter les shifts qui lui sont associés.	Moyenne
US8	En tant qu'Employee, je souhaite demander un congé afin de soumettre une période d'absence à l'administration.	La demande est enregistrée avec le statut PENDING après vérification des règles liées au contrat et au solde disponible.	Haute
US9	En tant que système, je souhaite calculer automatiquement les jours ouvrés d'un congé afin de déterminer la durée réelle de l'absence.	Les samedis et dimanches ne sont pas comptabilisés dans le nombre de jours de congé.	Haute
US10	En tant qu'Admin, je souhaite approuver ou rejeter une demande de congé afin de gérer les absences des employés.	Une demande PENDING peut être approuvée ou rejetée. En cas d'approbation, le solde de congés est automatiquement diminué.	Haute
US11	En tant qu'Employee, je souhaite demander une avance sur salaire afin de bénéficier d'une partie de mon salaire avant son versement.	Le montant demandé ne dépasse pas 50% du salaire et le cumul mensuel des avances approuvées respecte également cette limite.	Haute

US12	En tant qu'Admin, je souhaite approuver ou rejeter une demande d'avance sur salaire afin de gérer les demandes financières des employés.	Une demande PENDING peut être approuvée ou rejetée et son statut est mis à jour.	Haute
US13	En tant que système, je souhaite notifier l'administration lorsqu'une demande de congé est créée afin d'assurer son suivi.	Une notification est envoyée au service n8n après l'enregistrement de la demande de congé.	Moyenne

TABLE 6.1 – Sprint Backlog - Gestion des Ressources Humaines

6.3 Conception

Cette section présente la conception détaillée des fonctionnalités développées durant le quatrième sprint.

Elle décrit les interactions entre les différents acteurs du système ainsi que la structure des principales entités utilisées pour la gestion des ressources humaines.

6.3.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation présente les principales fonctionnalités du module de gestion des ressources humaines.

Il décrit les interactions entre l'administrateur, les employés et les différents services du système intervenant dans la gestion des horaires, des shifts, des congés, des avances sur salaire et de la vérification des employés.

Le système implique principalement les acteurs suivants :

- **Admin** : gère les horaires, les shifts, les demandes de congé, les demandes d'avance sur salaire et la vérification des employés.
- **Employee** : consulte ses horaires et ses shifts et peut effectuer des demandes de congé et d'avance sur salaire.
- **Service OCR** : intervient dans l'extraction automatique du numéro CIN à partir des documents d'identité.
- **n8n** : intervient dans le processus de notification des demandes de congé.

Les principaux cas d'utilisation réalisés dans ce sprint sont :

- Vérifier un employé par CIN
- Gérer les horaires de travail

- Créer un shift
- Affecter automatiquement un employé
- Consulter les shifts
- Demander un congé
- Approuver un congé
- Rejeter un congé
- Demander une avance sur salaire
- Approuver une avance
- Rejeter une avance

Description des Cas d'Utilisation Principaux

UC1 : Vérifier un employé par CIN Description :

Ce cas d'utilisation permet à l'Admin de vérifier l'identité d'un employé à partir des documents CIN fournis. Le système utilise un service OCR afin d'extraire automatiquement le numéro CIN.

Préconditions :

- L'Admin est authentifié.
- Les images recto et verso du CIN sont disponibles.
- Le service OCR est accessible.

Scénario nominal :

1. L'Admin sélectionne les images recto et verso du CIN.
2. Le système vérifie les fichiers transmis.
3. Les documents sont sauvegardés.
4. Le système transmet l'image au service OCR.
5. Le service OCR extrait le numéro CIN.
6. Le système recherche l'employé correspondant au CIN.
7. Le système marque l'employé comme vérifié.
8. Une notification de vérification est envoyée à l'administration.

Postconditions :

- L'employé est marqué comme vérifié.
- Le résultat de la vérification est retourné au client.

UC2 : Affecter automatiquement un employé Description :

Ce cas d'utilisation permet au système de sélectionner automatiquement un employé disponible pour un créneau donné.

Préconditions :

- Le département concerné existe.

- Le créneau demandé possède une date et des heures définies.
- Des horaires de travail sont configurés pour les employés.

Scénario nominal :

1. Le système récupère les employés du département concerné.
2. Il vérifie que leur horaire habituel couvre le créneau demandé.
3. Les employés inactifs ou indisponibles sont exclus.
4. Le système vérifie l'absence de conflit avec les shifts existants.
5. Un employé disponible est sélectionné.
6. Le système crée automatiquement un WorkShift pour bloquer le créneau.

Postconditions :

- Un employé est affecté au créneau.
- Un shift est créé avec le statut PLANNED.

UC3 : Demander un congé Description :

Ce cas d'utilisation permet à un Employee de soumettre une demande de congé. Le système vérifie les règles applicables selon le type de contrat ainsi que le solde de congés disponible.

Préconditions :

- L'Employee possède un contrat actif.
- Aucun autre congé n'est déjà en attente.
- Le solde de congés est suffisant.

Scénario nominal :

1. L'Employee sélectionne les dates de début et de fin.
2. Il saisit le motif du congé.
3. Le système calcule le nombre de jours ouvrés.
4. Le système vérifie les règles liées au type de contrat.
5. Le système vérifie le solde disponible.
6. La demande est enregistrée avec le statut PENDING.
7. Le système transmet les informations de la demande au service n8n.

Postconditions :

- La demande de congé est enregistrée.
- Son statut est PENDING.
- Une notification est déclenchée.

UC4 : Approuver ou rejeter une demande de congé Description :

Ce cas d'utilisation permet à l'Admin de traiter une demande de congé soumise par un employé.

Préconditions :

- La demande existe.
- La demande possède le statut PENDING.

Scénario nominal :

1. L'Admin consulte les demandes de congé.
2. Il sélectionne une demande en attente.
3. Il choisit de l'approuver ou de la rejeter.
4. En cas d'approbation, le système vérifie le solde disponible.
5. Le système met à jour le statut de la demande.
6. En cas d'approbation, le nombre de jours utilisés est déduit du solde.

Postconditions :

- La demande est APPROVED ou REJECTED.
- En cas d'approbation, le solde de congés est mis à jour.

UC5 : Demander une avance sur salaire Description :

Ce cas d'utilisation permet à un Employee de demander une avance sur salaire dans la limite autorisée par son salaire et les avances déjà approuvées durant le mois.

Préconditions :

- L'Employee possède un contrat.
- Aucune demande d'avance n'est déjà en attente.
- Le montant demandé respecte la limite autorisée.

Scénario nominal :

1. L'Employee saisit le montant de l'avance souhaitée.
2. Le système récupère le salaire du contrat.
3. Le système calcule la limite maximale correspondant à 50% du salaire.
4. Le système calcule les avances approuvées durant le mois.
5. Le système vérifie le montant demandé.
6. La demande est enregistrée avec le statut PENDING.

Postconditions :

- La demande d'avance est enregistrée.
- Son statut est PENDING.
- Le montant restant autorisé pour le mois est calculé.

6.3.2 Diagramme de Séquence : Affectation automatique d'un employé

Le diagramme de séquence suivant illustre le processus d'affectation automatique d'un employé à un créneau de travail.

Cette fonctionnalité constitue un processus métier important du module RH puisqu'elle permet au système de sélectionner automatiquement un employé en fonction de plusieurs conditions de disponibilité.

Le scénario principal est le suivant :

- Le système reçoit une demande d'affectation avec un département, une date et un créneau horaire.
- Le service récupère les horaires habituels correspondant au jour demandé.
- Les employés dont les horaires couvrent le créneau sont sélectionnés.
- Les employés inactifs ou indisponibles sont exclus.
- Le système vérifie les conflits avec les shifts existants.
- Le premier employé satisfaisant les conditions est sélectionné.
- Le système crée automatiquement un WorkShift.
- L'employé sélectionné est retourné au système.

6.3.3 Diagramme de Classes

Le diagramme de classes présente la structure statique du module de gestion des ressources humaines.

Il décrit les principales entités manipulées par le système ainsi que les relations permettant d'assurer la gestion des employés, des contrats, des horaires, des shifts, des congés et des avances sur salaire.

Les principales classes représentées sont :

- **Employee** : représente l'employé et contient ses informations personnelles ainsi que son état de disponibilité et de vérification.
- **Department** : représente le département auquel l'employé est associé.
- **Contract** : représente le contrat de travail de l'employé et contient notamment le type de contrat et le salaire.
- **WorkingHours** : représente les horaires habituels de travail d'un employé.
- **WorkShift** : représente un créneau de travail planifié pour un employé.
- **LeaveRequest** : représente une demande de congé soumise par un employé.
- **LeaveBalance** : représente le solde de congés disponible pour un employé.
- **SalaryAdvance** : représente une demande d'avance sur salaire.

Les relations principales sont :

- Un employé peut être associé à un département.
- Un employé peut posséder un ou plusieurs contrats.

- Un employé peut avoir plusieurs horaires de travail.
- Un employé peut posséder plusieurs shifts.
- Un employé peut effectuer plusieurs demandes de congé.
- Un employé possède un solde de congés.
- Un employé peut effectuer plusieurs demandes d’avance sur salaire.
- Un shift est associé à un employé et à un département.

6.4 Architecture et intégration des services

La gestion des ressources humaines repose sur plusieurs services métier permettant de séparer les responsabilités liées aux différentes fonctionnalités du module.

Le module est principalement organisé autour des services de gestion des horaires, des shifts, des congés et des avances sur salaire.

6.4.1 Gestion des horaires et des shifts

La gestion des horaires est assurée par le service *WorkingHoursService*. Celui-ci permet d’enregistrer, de modifier, de supprimer et de consulter les horaires de travail des employés.

Lorsqu’un horaire est enregistré, il est associé à un employé, un département, un jour de la semaine ainsi qu’à une heure de début et une heure de fin.

Le service *WorkShiftService* permet quant à lui de gérer les créneaux de travail planifiés. Il intègre également un mécanisme d’affectation automatique basé sur quatre conditions principales :

1. l’horaire habituel de l’employé doit couvrir le créneau demandé ;
2. l’employé doit être actif ;
3. l’employé doit être disponible ;
4. l’employé ne doit pas avoir de conflit avec un shift existant.

Lorsqu’un employé répond à ces conditions, un nouveau shift est automatiquement créé afin de réserver le créneau.

6.4.2 Gestion des congés

La gestion des congés est assurée par le service *LeaveService*.

Avant l’enregistrement d’une demande, le système vérifie l’existence d’un contrat actif, l’absence d’une autre demande en attente et la disponibilité du solde de congés.

Les règles appliquées dépendent du type de contrat :

- **CDI** : maximum de 25 jours et préavis minimum de 3 jours ;
- **CDD** : maximum de 15 jours, préavis minimum de 2 jours et respect de la date de fin du contrat ;

— **STAGE** : maximum de 5 jours et préavis minimum d'un jour.

Le système calcule également les jours ouvrés en excluant les samedis et les dimanches.

Après l'enregistrement de la demande, les informations sont transmises au service n8n afin de déclencher le mécanisme de notification.

6.4.3 Gestion des avances sur salaire

La gestion des avances est assurée par le service *SalaryAdvanceService*.

Lorsqu'un employé demande une avance, le système récupère son contrat et son salaire afin de déterminer le montant maximal autorisé.

La limite est fixée à 50% du salaire. Le système vérifie également le cumul des avances déjà approuvées durant le mois en cours afin de garantir que le plafond mensuel ne soit pas dépassé.

6.5 Réalisation

Cette section présente les principales fonctionnalités développées au cours de ce sprint concernant la gestion des ressources humaines.

Elle décrit les différentes interfaces réalisées pour la vérification des employés, la gestion des horaires et des shifts ainsi que la gestion des congés et des avances sur salaire.

6.5.1 Vérification d'un Employee par CIN

La fonctionnalité de vérification permet à l'administrateur de vérifier l'identité d'un employé à partir de son document CIN.

L'administrateur fournit les images recto et verso du document. Le système sauvegarde les fichiers puis transmet l'image au service OCR afin d'extraire automatiquement le numéro CIN.

Après extraction, le système recherche l'employé correspondant dans la base de données. Si le CIN est reconnu et qu'un employé correspondant existe, son statut de vérification est mis à jour.

Le système retourne également le niveau de confiance fourni par le service OCR. Dans le cas d'un résultat présentant une faible confiance, un avertissement est affiché afin d'encourager une vérification manuelle.

6.5.2 Gestion des horaires de travail

L'interface de gestion des horaires permet à l'administrateur de définir les horaires habituels des employés.

Pour chaque horaire, le système enregistre le jour de la semaine, l'heure de début, l'heure de fin et le département concerné.

Ces informations sont ensuite utilisées par le mécanisme d'affectation automatique afin de déterminer les employés pouvant être affectés à un créneau donné.

6.5.3 Gestion des shifts

L'interface de gestion des shifts permet de consulter et de planifier les créneaux de travail des employés.

Chaque shift est associé à une date, une heure de début, une heure de fin, un employé et un département. Lorsqu'un shift est créé automatiquement, son statut initial est défini à *PLANNED*.

Le système empêche également l'affectation d'un employé à plusieurs shifts présentant un chevauchement sur la même période.

6.5.4 Gestion des demandes de congé

L'employé peut soumettre une demande de congé en indiquant la période souhaitée ainsi que le motif de son absence.

Avant d'enregistrer la demande, le système applique automatiquement les règles correspondant au type de contrat de l'employé et vérifie son solde disponible.

Une demande valide est enregistrée avec le statut *PENDING*. L'administrateur peut ensuite consulter les demandes et décider de les approuver ou de les rejeter.

6.5.5 Gestion des avances sur salaire

L'employé peut également effectuer une demande d'avance sur salaire en indiquant le montant souhaité.

Le système récupère automatiquement le salaire correspondant au contrat de l'employé et vérifie que le montant demandé respecte la limite de 50% du salaire.

Le cumul des avances approuvées durant le mois est également pris en compte. Une demande dépassant le plafond autorisé est automatiquement refusée par le système.

L'administrateur peut ensuite approuver ou rejeter les demandes enregistrées.

6.6 Bilan du Sprint

Ce quatrième sprint a permis de mettre en place les principales fonctionnalités nécessaires à la gestion des ressources humaines au sein de l'application MSG.

Les fonctionnalités développées permettent notamment de vérifier l'identité des employés à l'aide d'un service OCR, de gérer leurs horaires et leurs shifts, ainsi que de suivre leurs demandes de congé et d'avance sur salaire.

Le mécanisme d'affectation automatique améliore également la gestion des ressources en sélectionnant un employé en fonction de son horaire, de son état d'activité, de sa disponibilité et de l'absence de conflit avec les shifts existants.

La gestion des congés intègre plusieurs règles métier adaptées aux différents types de contrats et permet de maintenir automatiquement le solde disponible après approbation d'une demande. De même, la gestion des avances sur salaire garantit le respect d'un plafond mensuel correspondant à 50% du salaire.

Enfin, l'intégration avec le service OCR et le workflow n8n permet d'automatiser certaines opérations du module RH, notamment la vérification des employés et la notification des demandes de congé.

Ainsi, ce sprint constitue une étape importante dans la mise en place d'un module RH complet, permettant d'améliorer le suivi des employés, l'organisation de leur temps de travail et la gestion des demandes administratives.

Chapitre 7

Intégration et automatisation avec n8n

7.1 Introduction

Ce sprint est consacré à l'intégration de la plateforme n8n avec le backend de l'application afin d'automatiser plusieurs processus métier. Contrairement aux sprints précédents, ce sprint ne porte pas sur la création d'un nouveau module fonctionnel, mais sur la mise en place d'un mécanisme d'automatisation permettant au système de déclencher des workflows externes à partir de différents événements.

L'intégration est réalisée entre le backend Spring Boot et n8n à travers des Webhooks HTTP. Lorsqu'un événement métier se produit dans l'application, le backend prépare les informations nécessaires et les transmet au workflow n8n correspondant.

Les principaux événements pris en charge concernent la confirmation des commandes, les paiements de maintenance, les situations de stock faible, la réception des livraisons ainsi que les demandes de congé.

7.2 Présentation et analyse du Sprint

7.2.1 Présentation de n8n

n8n est une plateforme d'automatisation permettant de créer et d'exécuter des workflows reliant différentes applications et services. Elle permet notamment de recevoir des données à travers des Webhooks, de les traiter et de déclencher automatiquement différentes actions selon le workflow configuré.

Dans notre application, n8n est utilisé comme un composant externe chargé d'exécuter des traitements d'automatisation à partir d'événements générés par le backend Spring Boot. La communication entre les deux systèmes est réalisée à travers des requêtes HTTP envoyées vers des Webhooks n8n.

Cette approche permet de séparer la logique métier principale de l'application des traitements d'automatisation. Ainsi, les workflows peuvent être modifiés ou enrichis indé-

pendamment du code principal du backend.

7.2.2 Objectifs du Sprint

L'objectif principal de ce sprint est d'intégrer n8n au backend afin d'automatiser plusieurs processus déclenchés par des événements métier.

Les objectifs réalisés durant ce sprint sont les suivants :

- intégrer n8n avec le backend Spring Boot ;
- mettre en place des Webhooks pour recevoir les événements provenant de l'application ;
- automatiser la notification après la confirmation d'une commande ;
- automatiser la notification après un paiement de maintenance ;
- générer des alertes lorsqu'un produit atteint un niveau de stock faible ;
- automatiser la notification lors de la réception d'une livraison ;
- transmettre automatiquement les informations relatives aux demandes de congé.

7.2.3 Sprint Backlog

Le Sprint Backlog regroupe les fonctionnalités développées durant ce sprint pour mettre en place l'intégration entre le backend Spring Boot et n8n.

TABLE 7.1 – Sprint Backlog du Sprint 5

Fonctionnalité	Description	Priorité
Intégration de n8n	Mettre en place la communication entre Spring Boot et n8n à travers des Webhooks HTTP.	Haute
Confirmation de commande	Transmettre les informations d'une commande confirmée au workflow n8n et mettre à jour la caisse.	Haute
Paiement de maintenance	Transmettre les informations relatives au paiement d'une maintenance au workflow n8n.	Haute
Alerte de stock faible	Détecter les produits dont le stock est faible et transmettre leurs informations à n8n.	Haute
Réception de livraison	Transmettre les informations relatives à une livraison reçue et mettre à jour la caisse.	Haute
Demande de congé	Transmettre automatiquement les informations d'une demande de congé à n8n.	Moyenne

7.3 Intégration de n8n

7.3.1 Architecture de l'intégration

L'intégration repose sur une communication entre le backend Spring Boot et la plateforme n8n. Le backend reste responsable de la logique métier et détecte les événements nécessitant une automatisation. Le service `N8nNotifierService` assure ensuite la préparation et l'envoi des données vers le Webhook n8n correspondant.

La communication peut être représentée de manière générale comme suit :

Backend Spring Boot $\rightarrow$ *N8nNotifierService* $\rightarrow$ *Webhook n8n* $\rightarrow$ *Workflow n8n* $\rightarrow$
Action automatisée

7.3.2 Communication entre Spring Boot et n8n

La communication entre les deux systèmes est réalisée à l'aide de requêtes HTTP. Pour cela, le service `N8nNotifierService` utilise la classe `RestTemplate` de Spring afin d'envoyer les données au Webhook correspondant.

Chaque type d'événement possède une URL de Webhook configurée dans les propriétés de l'application. Le service récupère ces URLs à l'aide de l'annotation `@Value`.

Par exemple, le service possède plusieurs URLs correspondant aux différents événements :

```
n8n.webhook.order-confirmed
n8n.webhook.low-stock
n8n.webhook.supply-received
n8n.webhook.leave-request
n8n.webhook.maintenance-payment
```

Lorsqu'un événement se produit, les données nécessaires sont regroupées dans une structure de type `Map<String, Object>`, puis envoyées au Webhook correspondant à l'aide de `RestTemplate`.

7.3.3 Gestion des Webhooks

Un Webhook est utilisé comme point d'entrée d'un workflow n8n. Lorsqu'un événement métier est détecté dans l'application, le backend envoie une requête HTTP POST vers le Webhook correspondant.

Le service `N8nNotifierService` centralise cette communication afin d'éviter de dupliquer la logique d'appel aux Webhooks dans les différents services métier.

Cinq types d'événements sont actuellement pris en charge :

- confirmation d'une commande ;

- paiement d’une maintenance ;
- détection d’un stock faible ;
- réception d’une livraison ;
- création d’une demande de congé.

Chaque événement possède son propre payload contenant les informations nécessaires au traitement du workflow n8n.

7.4 Réalisation

7.4.1 Automatisation de la confirmation des commandes

Lorsqu’une commande est confirmée, le backend déclenche la méthode `notifyOrderConfirmed`. Cette méthode réalise d’abord l’enregistrement de la vente dans la caisse à travers le service `CashRegisterService`.

Le montant total de la commande est crédité dans la caisse avec la catégorie `SALE_CASH`. Le nouveau solde de la caisse est ensuite récupéré afin de l’intégrer dans les données envoyées à n8n.

Le payload transmis contient notamment :

- l’identifiant de la commande ;
- le nom du client ;
- le montant total ;
- le statut de la commande ;
- le nouveau solde de la caisse.

Le payload est ensuite envoyé au Webhook associé à la confirmation de commande.

7.4.2 Automatisation des paiements de maintenance

Lorsqu’un paiement relatif à une maintenance est effectué, la méthode `notifyMaintenancePayment` est appelée.

Le service récupère le nouveau solde de la caisse et prépare un payload contenant les informations du paiement. Ces informations comprennent notamment le numéro de facture, le montant payé, le ou les moyens de paiement utilisés, la date du paiement ainsi que le solde actuel de la caisse.

Les données sont ensuite transmises au Webhook n8n dédié aux paiements de maintenance.

7.4.3 Gestion des alertes de stock faible

Le système permet également d’automatiser la gestion des situations de stock faible.

La méthode `notifyLowStock` reçoit une liste de produits dont le niveau de stock nécessite une attention particulière. Pour chaque produit, les informations relatives au stock sont préparées avant d'être regroupées dans un payload.

Les données transmises comprennent notamment :

- le nom du produit ;
- la quantité disponible ;
- le seuil de réapprovisionnement ;
- l'emplacement du stock ;
- le nombre de produits concernés ;
- la date et l'heure de la vérification.

Ces informations sont envoyées au Webhook n8n dédié à la gestion des stocks faibles.

7.4.4 Automatisation de la réception des livraisons

Lorsqu'une livraison est reçue, la méthode `notifySupplyReceived` est utilisée pour transmettre les informations relatives à l'opération à n8n.

Avant l'envoi du payload, le coût total de la réception est calculé à partir de la quantité reçue et du prix unitaire. Le montant correspondant est ensuite débité de la caisse avec la catégorie `PURCHASE_CASH`.

Le payload envoyé à n8n contient notamment :

- le nom et l'identifiant du produit ;
- le nom et l'identifiant du fournisseur ;
- la quantité reçue ;
- le prix unitaire ;
- le coût total ;
- une éventuelle note ;
- la nouvelle quantité disponible en stock ;
- la date de réception ;
- le nouveau solde de la caisse.

Le Webhook n8n correspondant reçoit ces informations afin de déclencher le workflow associé à la réception de la livraison.

7.4.5 Notification des demandes de congé

L'intégration de n8n est également utilisée pour automatiser le traitement des demandes de congé.

Lorsqu'un employé soumet une demande, le backend sauvegarde la demande puis appelle la méthode `notifyLeaveRequest`. Cette méthode prépare les informations nécessaires et les transmet au Webhook n8n correspondant.

Le payload contient notamment :

- l’identifiant de la demande ;
- le nom de l’employé ;
- son identifiant Telegram ;
- la date de début et la date de fin du congé ;
- le motif de la demande ;
- le nombre de jours ouvrés ;
- le type de contrat ;
- le nombre de jours de congé restants ;
- la date et l’heure de création de la demande.

Ces informations permettent au workflow n8n de déclencher les actions d’automatisation configurées pour les demandes de congé.

7.5 Bilan du Sprint

Ce sprint a permis d’intégrer la plateforme n8n au backend Spring Boot afin d’automatiser plusieurs processus de l’application.

La mise en place du service `N8nNotifierService` permet de centraliser la communication avec les différents Webhooks et d’assurer la transmission des données nécessaires aux workflows n8n.

L’intégration couvre plusieurs domaines de l’application, notamment les commandes, les paiements de maintenance, la gestion du stock, les approvisionnements et la gestion des ressources humaines.

Cette approche permet de réduire le traitement manuel de certaines opérations et offre une architecture plus flexible pour l’ajout de nouveaux workflows d’automatisation.

Conclusion Générale

Votre conclusion.

Bibliographie

- [1] VMware, “Spring security reference documentation,” 2024. [Online]. Available : <https://docs.spring.io/spring-security/reference/>
- [2] C. Walls, *Spring in Action*, 6th ed. Manning Publications, 2022.
- [3] M. Jones, J. Bradley, and N. Sakimura, “Rfc 7519 : Json web token (jwt),” 2015. [Online]. Available : <https://www.rfc-editor.org/rfc/rfc7519>